MODUL PRAKTIIKUM 04

FORM, INTERAKSI, DAN INTEGRASI PUSTAKA

TextField • Checkbox • RadioButton
ElevatedButton • FloatingActionButton • GestureDetector
Pustaka Chat • Google Maps



Mata Kuliah	Pemrograman Perangkat Bergerak Lanjut
Modul	IV – Form, Interaksi, dan Integrasi Pustaka
Tahun	2026

Modul praktikum ini digunakan sebagai panduan kegiatan praktikum mahasiswa.

DAFTAR ISI

A. Tujuan Pembelajaran 4
B. Tugas Pendahuluan 4
C. Dasar Teori 4
a. Mengenal Form dan Input pada Flutter 5
b. *TextField* 5
c. *Checkbox* 6
d. *RadioButton* 7
e. *ElevatedButton* 8
f. *FloatingActionButton* 9
g. *GestureDetector* 10
h. Pustaka *Chat* 11
i. Google Maps 12
j. Pustaka *Chart* 13
D. Alat dan Bahan 14
a. Perangkat Keras (Hardware) 14
b. Perangkat Lunak (*Software*) 14
E. Materi Praktikum 15
a. Menyiapkan Proyek Praktikum 15
b. Membuat Form dengan *TextField* 17
c. Menambahkan *Checkbox* 22
d. Menambahkan *RadioButton* 24
e. Menggunakan *ElevatedButton* untuk Mengirim Form 26
f. Menambahkan *FloatingActionButton* 29
g. Menggunakan *GestureDetector* untuk Interaksi Kustom 32
h. Mengintegrasikan Pustaka *Chat* Sederhana 36
i. Mengintegrasikan Google Maps 38
j. Mengintegrasikan Pustaka *Chart* 39
k. Menggabungkan Semua Fitur dengan Navigasi 41
F. Materi Praktikum dengan Prompt 45
a. Menyiapkan Proyek Praktikum dengan Bantuan AI 45
b. Membuat Form dan *TextField* dengan Bantuan AI 47
c. Menambahkan *Checkbox* dengan Bantuan AI 48
d. Menambahkan *RadioButton* dengan Bantuan AI 48
e. Menggunakan *ElevatedButton* dengan Bantuan AI 49
f. Menambahkan *FloatingActionButton* dengan Bantuan AI 50
g. Menggunakan *GestureDetector* dengan Bantuan AI 51
h. Mengintegrasikan Pustaka Chat dengan Bantuan AI 52
i. Mengintegrasikan Google Maps dengan Bantuan AI 53
j. Mengintegrasikan Pustaka Chart dengan Bantuan AI 54
k. Menggabungkan Semua Fitur dengan Navigasi dengan Bantuan AI 55
l. Menggabungkan, Menguji, dan Memperbaiki Galat dengan Bantuan AI 57

m. Mengevaluasi Jawaban AI 57
G. Tugas Praktikum 58
H. Format Pengumpulan Tugas 61

A. Tujuan Pembelajaran

Setelah mengikuti praktikum ini, mahasiswa diharapkan mampu:

- Membuat form input menggunakan *TextField* beserta *TextEditingController* dan validasi sederhana.
- Menggunakan *Checkbox* untuk merekam pilihan ya/tidak dan menyimpannya sebagai *state*.
- Menggunakan *RadioButton* untuk memilih satu opsi dari beberapa pilihan yang saling eksklusif.
- Menjalankan aksi menggunakan *ElevatedButton* serta menampilkan umpan balik atas aksi tersebut.
- Menambahkan *FloatingActionButton* untuk aksi utama yang mudah dijangkau pengguna.
- Mendeteksi interaksi sentuh kustom seperti tap, double tap, dan long press melalui *GestureDetector*.
- Menyusun tampilan percakapan sederhana menggunakan pustaka *chat* untuk menampilkan daftar pesan.
- Menampilkan peta dan penanda lokasi menggunakan pustaka Google Maps pada aplikasi Flutter.

B. Tugas Pendahuluan

1. Jelaskan alur pembaruan state dari widget input pada Flutter, mulai dari pengguna mengetik atau memilih hingga tampilan diperbarui! Mengapa nilai input sebaiknya disimpan di luar widget input melalui setState() atau Provider?
2. Apa fungsi TextEditingController pada TextField? Jelaskan perbedaan membaca nilai teks melalui controller dan melalui callback onChanged, serta peran properti decoration (InputDecoration) dalam menampilkan label dan pesan validasi!
3. Jelaskan perbedaan Checkbox dan RadioButton berdasarkan jenis pilihan yang ditanganinya, serta berikan masing-masing satu contoh penggunaan yang tepat pada sebuah formulir pendaftaran!
4. Apa perbedaan ElevatedButton dan FloatingActionButton dilihat dari fungsi dan penempatannya pada Scaffold? Berikan contoh aksi yang tepat untuk masing-masing tombol tersebut!
5. Apa yang dimaksud dengan GestureDetector, dan kapan widget ini lebih tepat digunakan dibandingkan tombol bawaan? Jelaskan perbedaan callback onTap, onDoubleTap, dan onLongPress!
6. Jelaskan peran pustaka pihak ketiga pada aplikasi Flutter, khususnya pustaka chat, Google Maps, dan pustaka chart! Sebutkan komponen utama tampilan percakapan, persiapan yang diperlukan agar peta Google Maps dapat ditampilkan, serta contoh jenis grafik yang dapat dibuat menggunakan pustaka chart!

C. Dasar Teori

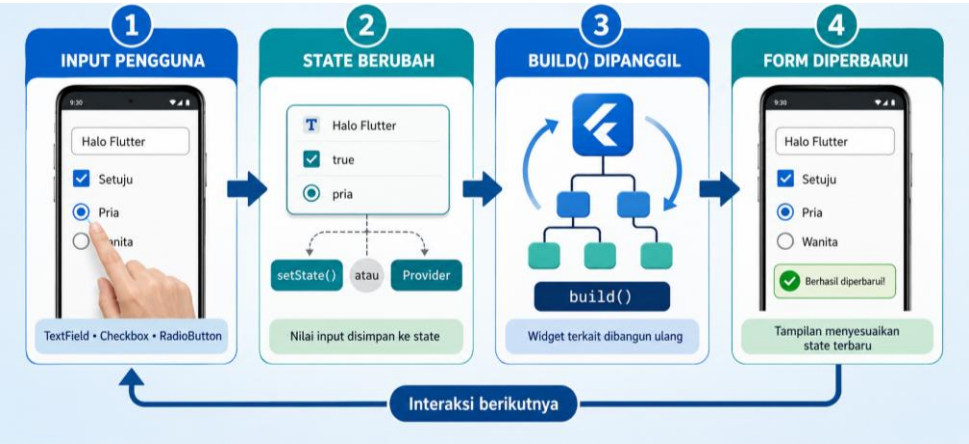
Antarmuka Flutter yang baik tidak hanya menampilkan data secara reaktif, tetapi juga menerima masukan dari pengguna dan merespons sentuhan dengan tepat. Bagian ini melanjutkan materi pengelolaan *state* pada Modul III dengan membahas *widget* interaksi pengguna: form dan input (*TextField*, *Checkbox*, *RadioButton*), *widget* aksi (*ElevatedButton*, *FloatingActionButton*,

GestureDetector), serta dua pustaka pihak ketiga yang sering dipakai pada aplikasi nyata, yaitu pustaka *chat* dan Google Maps.

setiap *widget* input menyimpan nilai sementara sebagai *state*, dan setiap *widget* aksi memicu perubahan *state* atau menjalankan suatu perintah. Semua *widget* pada bagian ini tetap memerlukan *setState*() atau *Provider*, sesuai dengan Modul III, untuk benar-benar memperbarui tampilan setelah pengguna berinteraksi.

a. Mengenal Form dan Input pada Flutter

Form adalah kumpulan *widget* input yang menampung data dari pengguna, misalnya nama, jumlah, atau pilihan. Flutter tidak memaksa penggunaan *widget* Form, tetapi menyediakannya untuk memvalidasi banyak input sekaligus. Setiap *widget* input, seperti *TextField*, *Checkbox*, dan *RadioButton*, menyimpan nilainya sendiri sebagai *state* lokal. Nilai tersebut harus dibaca oleh kode program dan disimpan kembali, biasanya melalui *setState*() atau melalui model yang dikelola *Provider*, agar tidak hilang ketika *widget* dibangun ulang.



Gambar 2.1 Alur sederhana pembaruan state dari widget input pada Flutter

Alur umumnya: Pengguna mengetik atau memilih pada *widget* input → *callback* (*onChanged*, *onTap*, *onPressed*) dipanggil → nilai baru disimpan melalui *setState*() atau model *Provider* → *build*() dipanggil ulang → tampilan menampilkan nilai terbaru. Karena itu, *widget* input sebaiknya hanya melaporkan perubahan melalui *callback*, sedangkan penyimpanan nilai dilakukan di luar *widget* tersebut.

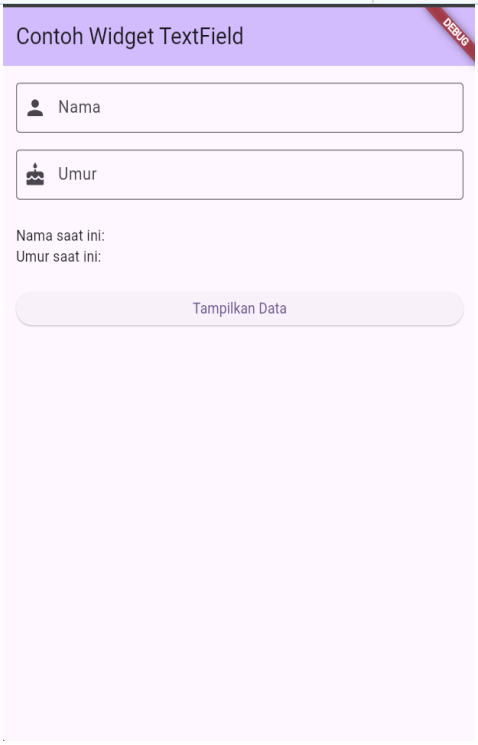
b. *TextField*

TextField adalah *widget* input teks satu baris atau beberapa baris. Nilai yang diketik pengguna dibaca melalui *TextEditingController* atau melalui *callback onChanged*. Tampilan *TextField*, seperti *label* dan pesan validasi, diatur melalui properti *decoration* bertipe *InputDecoration*.

Tabel 2.1 Properti Penting pada TextField

Properti	Fungsi	Contoh Nilai
controller	Membaca dan mengatur nilai teks dari luar *widget*.	*TextEditingController*()

Properti	Fungsi	Contoh Nilai
decoration	Mengatur *label*, *hint*, ikon, dan pesan galat pada kotak input.	*InputDecoration*(labelText: 'Nama')
keyboardType	Menentukan jenis papan ketik yang tampil.	*TextInputType*.number
onChanged	Callback yang dipanggil setiap kali teks berubah.	(*value*) => *setState*(() { nama = *value*; })



Gambar 2.2 Ilustrasi tampilan TextField dengan label dan teks yang sedang diketik pengguna

c. *Checkbox*

Checkbox adalah *widget* input biner yang menyimpan nilai benar atau salah (*bool*). *Checkbox* tidak mengelola *state*-nya sendiri: nilai yang tampil ditentukan oleh properti *value*, dan setiap sentuhan pengguna hanya memicu *callback onChanged* tanpa mengubah tampilan secara otomatis. Nilai baru harus disimpan kembali, biasanya melalui *setState*(), agar tanda centang benar-benar berubah.

Tabel 2.2 Properti Penting pada Checkbox

Properti	Fungsi	Contoh Nilai	Catatan
value	Menyimpan status tercentang atau tidak.	*true* / *false*	Wajib diisi, bertipe *bool*.
onChanged	Callback ketika pengguna menekan checkbox.	(v) => *setState*(() { setuju = v ?? *false*; })	Jika diberi *null*, checkbox menjadi nonaktif.



Gambar 2.3 Ilustrasi tampilan Checkbox pada kondisi tercentang dan tidak tercentang

d. *RadioButton*

RadioButton (*widget Radio*) dipakai ketika pengguna harus memilih satu opsi dari beberapa pilihan yang saling eksklusif, misalnya metode pembayaran. Setiap *Radio* membaca dua nilai: *value* milik dirinya sendiri dan *groupValue* yang dibagikan oleh seluruh *Radio* dalam satu grup. *Radio* dianggap terpilih ketika *value* sama dengan *groupValue*. Sentuhan pengguna memicu *onChanged*, dan *groupValue* harus diperbarui melalui *setState*() agar pilihan yang tampil benar-benar berubah.

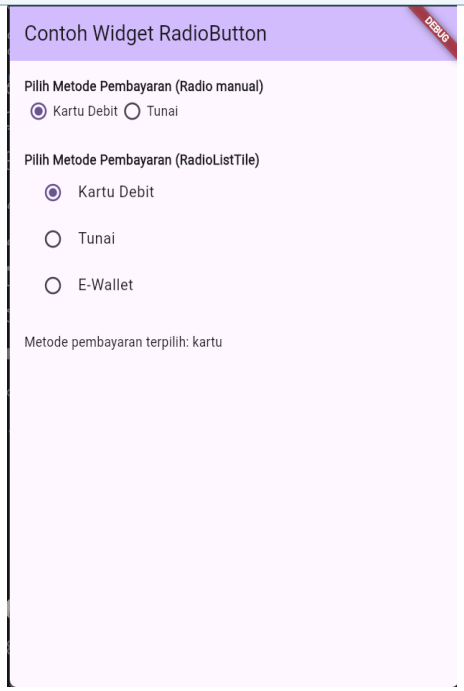
Contoh inti *RadioButton*

```
Radio<String>(
  value: 'kartu',
  groupValue: metodeBayar,
  onChanged: (v) => setState(() {
    metodeBayar = v;
  }),
)
```

- Seluruh *RadioButton* dalam satu grup harus berbagi *groupValue* yang sama, agar hanya satu pilihan yang aktif dalam satu waktu.
- Setiap *RadioButton* diberi *value* yang berbeda dari anggota grup lainnya, sehingga Flutter dapat membedakan opsi mana yang sedang dipilih.
- Gunakan *RadioListTile* ketika *label* perlu ditampilkan berdampingan dengan tombol radio, tanpa menyusun *Row* secara manual.

Contoh *RadioListTile* dengan *label*

```
RadioListTile<String>(
  title: const Text('Kartu Debit'),
  value: 'kartu',
  groupValue: metodeBayar,
  onChanged: (v) => setState(() {
    metodeBayar = v;
  }),
)
```



Gambar 2.4 Ilustrasi tampilan RadioButton dengan beberapa pilihan dan satu pilihan yang aktif

e. *ElevatedButton*

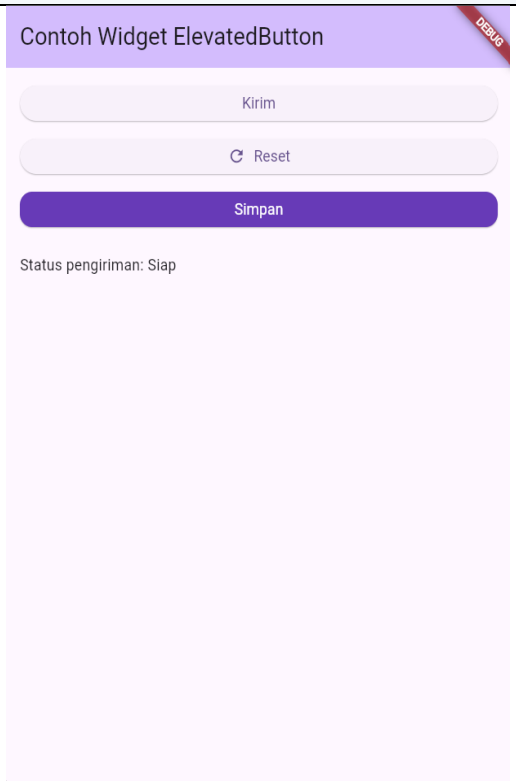
ElevatedButton adalah *widget* aksi utama bergaya terangkat, umum dipakai untuk aksi penting seperti mengirim form atau menyimpan data. Aksi yang dijalankan ditentukan oleh properti *onPressed* bertipe *VoidCallback*. Jika *onPressed* diberi nilai *null*, *ElevatedButton* otomatis tampil nonaktif (berwarna abu-abu) tanpa perlu logika tambahan.

- Properti *onPressed* menerima *VoidCallback* tanpa argumen; isi dengan *null* untuk menonaktifkan tombol tanpa menyembunyikannya dari tampilan.
- Properti *child* biasanya berupa Text, atau *Row* berisi *Icon* dan Text, untuk menampilkan *label* pada permukaan tombol.
- Properti *style* menerima *ElevatedButton*.styleFrom(...) untuk mengatur warna latar, warna teks, bentuk sudut, dan ukuran tombol.

Contoh inti *ElevatedButton*

```
ElevatedButton(
  onPressed: () {
    setState(() { sedangKirim = true; });
```

```
  },
  child: const Text('Kirim'),
)
```



Gambar 2.5 Ilustrasi tampilan ElevatedButton pada kondisi aktif dan nonaktif

f. *FloatingActionButton*

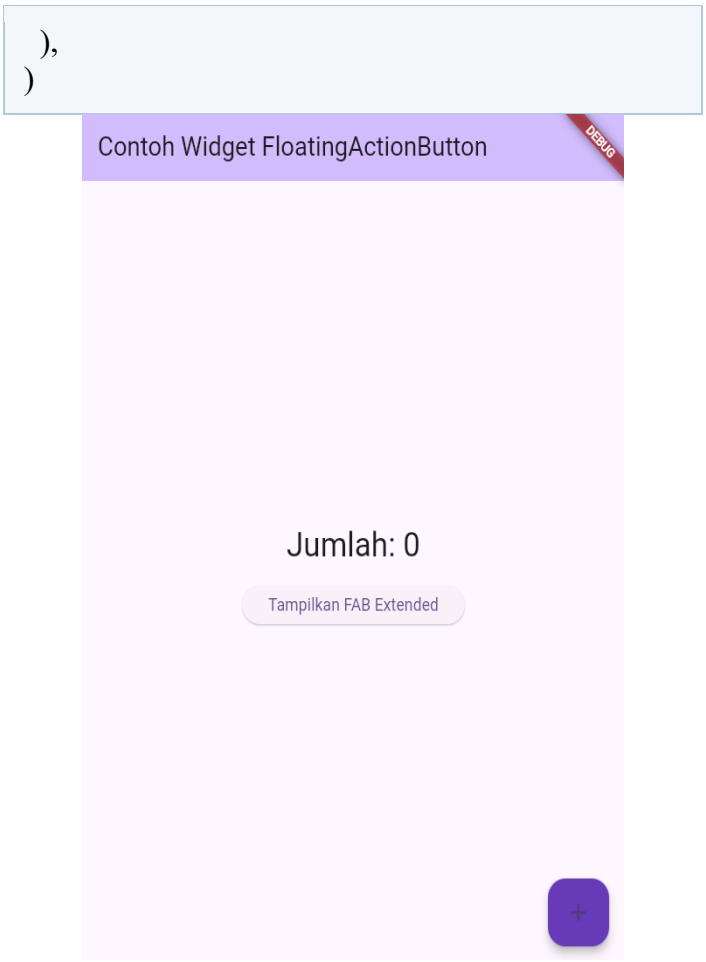
FloatingActionButton (FAB) adalah tombol bulat mengambang yang menonjolkan aksi utama pada suatu halaman, misalnya menambah item atau membuka obrolan baru. FAB biasanya dipasang melalui properti *floatingActionButton* pada *Scaffold* sehingga posisinya konsisten di kanan bawah tanpa perlu diatur manual.

Tabel 2.3 Properti dan Variasi FloatingActionButton

Bagian	Fungsi	Contoh
onPressed	Aksi yang dijalankan ketika FAB ditekan.	() => *setState*(() { jumlah++; })
child	Ikon yang tampil di tengah tombol.	*Icon*(Icons.add)
FloatingActionButton.extended	Variasi FAB dengan *label* teks di samping ikon.	icon: *Icon*(Icons.*chat*), *label*: Text('*Chat*')
backgroundColor	Mengubah warna latar tombol.	Colors.deepPurple

Contoh inti *FloatingActionButton*

```
Scaffold(
  floatingActionButton: FloatingActionButton(
    onPressed: () => setState(() { jumlah++; }),
    child: const Icon(Icons.add),
```



Gambar 2.6 Ilustrasi tampilan FloatingActionButton pada sudut layar aplikasi

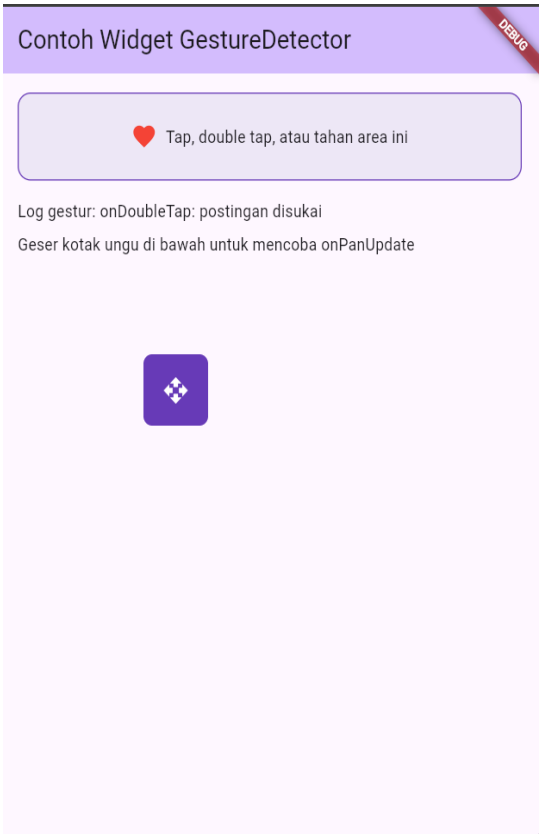
g. *GestureDetector*

GestureDetector adalah *widget* tak terlihat yang membungkus *widget* lain untuk mendeteksi berbagai jenis sentuhan yang tidak disediakan oleh tombol biasa, misalnya ketuk ganda, tahan lama, atau geser. Setiap jenis sentuhan memiliki *callback* tersendiri, dan hanya *callback* yang diisi yang akan aktif mendeteksi gestur tersebut.

Tabel 2.4 Callback Utama pada GestureDetector

Callback	Dipicu Oleh	Kapan Dipakai	Contoh
onTap	Satu sentuhan singkat.	Aksi sederhana, misalnya membuka halaman detail.	*onTap*: () => *Navigator*.push(*context*, ...)
onDoubleTap	Dua sentuhan cepat berurutan.	Aksi pintasan, misalnya menyukai sebuah postingan.	*onDoubleTap*: () => *setState*(() { disukai = *true*; })
onLongPress	Sentuhan yang ditahan beberapa saat.	Menampilkan menu tambahan atau dialog konfirmasi.	*onLongPress*: () => tampilkanMenu(*context*)

Callback	Dipicu Oleh	Kapan Dipakai	Contoh
onPanUpdate	Sentuhan yang digeser di layar.	Menggeser atau menggambar elemen secara langsung.	*onPanUpdate*: (d) => *setState*(() { posisi += d.delta; })



Gambar 2.7 Ilustrasi area sentuh GestureDetector dengan beberapa jenis gestur

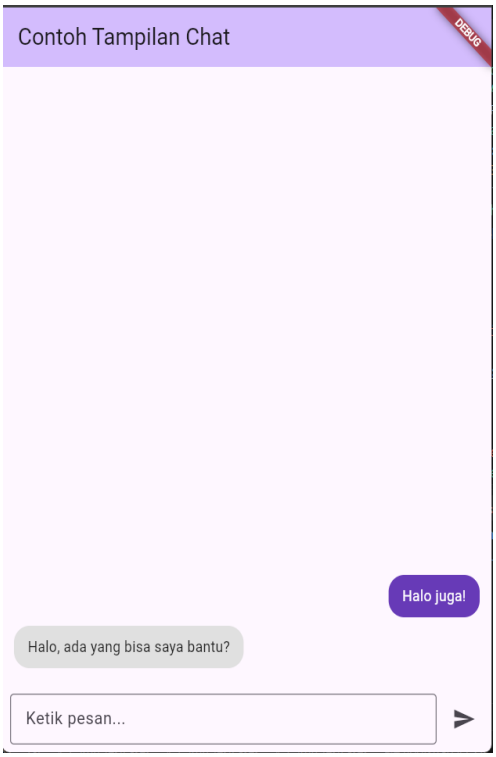
h. Pustaka *Chat*

Untuk membuat tampilan *chat* sendiri, dibutuhkan tiga bagian utama: daftar pesan yang bisa digulir, *chat bubble* dengan tampilan berbeda antara pengirim dan penerima, serta kolom input beserta tombol kirim. Jika tidak ingin membuat semuanya dari awal, pustaka pihak ketiga seperti *flutter_chat_ui* atau *dash_chat_2* sudah menyediakan *widget* siap pakai untuk kebutuhan tersebut. Meski begitu, konsep dasarnya tetap sama seperti *widget* Flutter lainnya: setiap pesan baru adalah perubahan *state*, dan *state* tersebut harus memicu *setState*() atau *notifyListeners*() agar daftar pesan tampil ulang.

Tabel 2.5 Bagian Utama Tampilan Chat

Bagian	Fungsi	Contoh Implementasi
Daftar pesan	Menampilkan riwayat percakapan, pesan terbaru berada di bagian bawah.	*ListView.builder*(reverse: *true*, itemCount: pesan.length, ...)
Gelembung pesan (*chat bubble*)	Membedakan pesan milik pengguna dan lawan bicara melalui warna serta posisi.	*Align*(alignment: kirim ? Alignment.centerRight : Alignment.centerLeft, ...)

Bagian	Fungsi	Contoh Implementasi
Kolom input pesan	Menampung teks yang sedang diketik sebelum dikirim.	*TextField*(*controller*: pesanController)
Tombol kirim	Menambahkan pesan baru ke daftar dan mengosongkan kolom input.	*IconButton*(icon: const *Icon*(Icons.send), *onPressed*: kirimPesan)



Gambar 2.8 Ilustrasi tampilan chat dengan gelembung pesan pengirim dan penerima

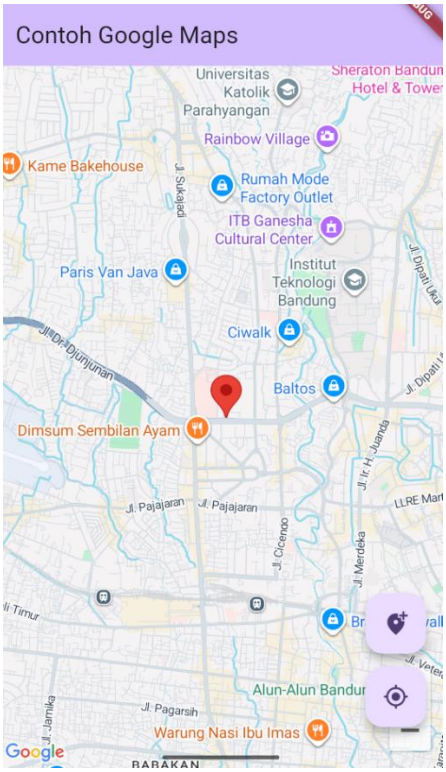
i. Google Maps

Google Maps dapat ditampilkan pada aplikasi Flutter melalui pustaka *google_maps_flutter*, yang membungkus komponen peta asli Android dan iOS. Peta ditampilkan lewat *widget GoogleMap* yang memerlukan posisi kamera awal (*CameraPosition* berisi titik *LatLng* dan tingkat *zoom*) serta kumpulan *Marker* untuk menandai lokasi penting. Sama seperti *widget* lain, setiap penanda yang ditambahkan, dihapus, atau dipindahkan adalah perubahan *state* yang harus melalui *setState*() atau model *Provider* agar peta pada layar diperbarui.

Tabel 2.6 Bagian Utama pada google_maps_flutter

Bagian	Fungsi	Contoh
GoogleMap	*Widget* utama yang menampilkan peta.	*GoogleMap*(*initialCameraPosition*: kamera, *markers*: *markers*)
CameraPosition	Menentukan titik tengah dan tingkat *zoom* awal peta.	*CameraPosition*(*target*: *LatLng*(-6.9, 107.6), *zoom*: 14)
Marker	Menandai satu lokasi pada peta beserta info singkat.	*Marker*(markerId: *MarkerId*('kampus'), position: *LatLng*(...), infoWindow: *InfoWindow*(title: 'Kampus'))

Bagian	Fungsi	Contoh
GoogleMapController	Mengatur peta secara terprogram, misalnya memindahkan kamera.	*controller*.animateCamera(CameraUpdate.newLatLng(titik))



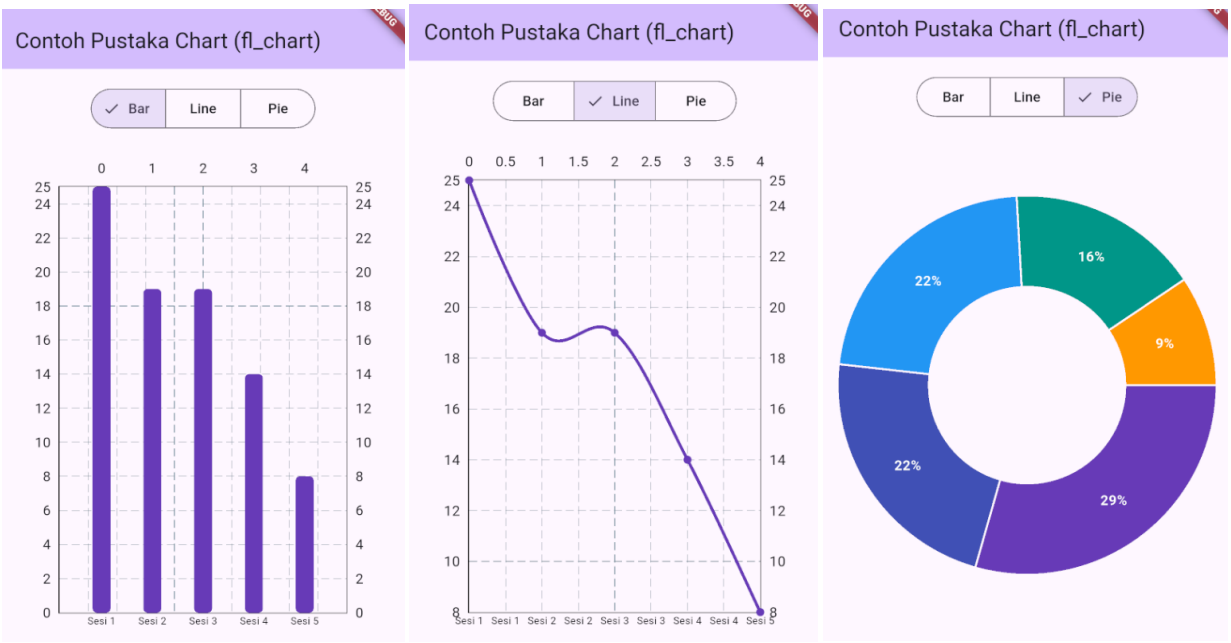
Gambar 2.9 Ilustrasi tampilan Google Maps dengan penanda lokasi

j. Pustaka *Chart*

Data yang berupa kumpulan angka, misalnya jumlah pendaftar pada setiap sesi kunjungan, lebih mudah dipahami ketika ditampilkan sebagai *chart* daripada sekadar daftar angka. Pustaka *fl_chart* menyediakan *widget chart* siap pakai seperti *BarChart*, *LineChart*, dan *PieChart* yang dapat disisipkan seperti *widget* Flutter lainnya. Sama seperti *widget* lain, data pada *chart* hanya berubah tampilannya setelah dipicu *setState*() atau *notifyListeners*(), tergantung apakah data dikelola sebagai *state* lokal atau melalui *Provider*.

Tabel 2.10 Bagian Utama pada *fl_chart*

Bagian	Fungsi	Contoh
BarChart	Menampilkan data sebagai batang, cocok untuk membandingkan jumlah antar kategori.	*BarChart*(*BarChartData*(barGroups: kelompokBatang))
LineChart	Menampilkan data sebagai garis, cocok untuk tren dari waktu ke waktu.	*LineChart*(LineChartData(lineBarsData: garisData))
PieChart	Menampilkan data sebagai proporsi lingkaran, cocok untuk perbandingan persentase.	*PieChart*(PieChartData(sections: bagianPie))



Gambar 2.10 Ilustrasi tampilan chart batang menampilkan jumlah pendaftar per sesi

D. Alat dan Bahan

a. Perangkat Keras (Hardware)

- Komputer atau laptop 64 bit dengan RAM minimal 8 GB; 16 GB disarankan apabila *emulator* dan Android Studio dijalankan bersamaan.
- Perangkat Android fisik dan kabel data sebagai alternatif *emulator*, dengan USB Debugging yang telah aktif.

b. Perangkat Lunak (*Software*)

- Flutter SDK pada kanal stabil dan Dart SDK yang dikenali oleh perintah flutter doctor.
- Android Studio atau Visual Studio Code beserta ekstensi Flutter dan Dart.
- Paket *provider*. Tambahkan melalui perintah flutter pub add *provider* untuk melanjutkan pola pengelolaan state *ChangeNotifier* dan *Provider* dari praktikum sebelumnya.
- Paket *google_maps_flutter*. Tambahkan melalui perintah flutter pub add *google_maps_flutter* agar versi yang sesuai dicatat otomatis pada pubspec.yaml.
- Paket *fl_chart.* Tambahkan melalui perintah flutter pub add *fl_chart* untuk menampilkan data sebagai *chart* batang, garis, atau lingkaran.
- Paket ikon tambahan (opsional) seperti *flutter_chat_ui* atau *dash_chat_2* apabila ingin memakai komponen *chat* siap pakai, atau cukup gunakan *ListView* dan *TextField* bawaan Flutter untuk membangun tampilan *chat* sendiri.
- Kunci API Google Maps (Maps SDK for Android/iOS) yang telah diaktifkan pada Google Cloud Console, didaftarkan pada AndroidManifest.xml (Android) dan *AppDelegate*.swift (iOS).
- Git dan akun repositori untuk menyimpan hasil praktikum.

E. Materi Praktikum

Bagian ini menuntun mahasiswa membangun halaman formulir pendaftaran kegiatan kampus secara bertahap: input teks, checkbox, radio button, tombol aksi, interaksi sentuhan kustom, tampilan *chat* sederhana, hingga peta lokasi kegiatan menggunakan Google Maps.

a. Menyiapkan Proyek Praktikum

Praktikum ini melanjutkan pendekatan pengelolaan state dengan *ChangeNotifier* dan *Provider* seperti pada praktikum sebelumnya. Buat proyek baru, tambahkan paket yang dibutuhkan, lalu siapkan folder lib/model/ untuk menampung data yang dipakai bersama oleh beberapa halaman.

```
flutter create form_pendaftaran_nim
cd form_pendaftaran_nim
flutter pub add provider
flutter pub add google_maps_flutter
```

Susun struktur berkas berikut agar setiap halaman mudah ditelusuri.

- lib/main.dart — titik masuk aplikasi dan navigasi antar halaman.
- lib/model/pendaftar_model.dart — model *ChangeNotifier* yang menyimpan daftar pendaftar beserta sesi yang dipilih, dipakai bersama oleh halaman formulir dan statistik.
- lib/pages/formulir_page.dart — halaman formulir pendaftaran (*TextField*, *Checkbox*, *RadioButton*, *ElevatedButton*, *FloatingActionButton*, *GestureDetector*).
- lib/pages/*chat*_page.dart — halaman *chat* sederhana antara peserta dan panitia.
- lib/pages/statistik_page.dart — halaman statistik jumlah pendaftar per sesi dalam bentuk *chart*.
- lib/pages/peta_page.dart — halaman peta lokasi kegiatan menggunakan Google Maps.
- lib/navigation/kerangka_navigasi.dart — kerangka navigasi utama yang menyatukan Formulir, Chat, Peta, dan Statistik menggunakan *NavigationBar* (layar sempit) atau *NavigationRail* (layar lebar), mengikuti pola Modul II.
- lib/pages/detail_kegiatan_page.dart — halaman detail yang dituju oleh *GestureDetector* pada kartu informasi kegiatan.
- lib/navigation/app_routes.dart — kelas *AppRoutes* yang menyimpan nama route sebagai konstanta dan mendaftarkan setiap halaman, mengikuti pola Modul II.

Aplikasi ini akan diuji pada Android maupun iOS, sehingga kunci API Google Maps perlu diperoleh dan dipasang pada kedua platform tersebut.

Langkah mendapatkan kunci API Google Maps:

- Buka Google Cloud Console (console.cloud.google.com), lalu buat proyek baru atau pilih proyek yang sudah ada.
- Pada menu APIs & Services > Library, aktifkan Maps SDK for Android dan Maps SDK for iOS untuk proyek tersebut.
- Buka APIs & Services > Credentials, pilih Create Credentials > API Key. Sebuah kunci API akan langsung dibuat.
- Batasi kunci API agar lebih aman: buat dua kunci terpisah, satu dibatasi untuk aplikasi Android (berdasarkan nama paket dan sidik jari sertifikat SHA-1), satu lagi dibatasi untuk aplikasi iOS (berdasarkan bundle identifier), lalu salin masing-masing kunci untuk dipasang pada langkah berikut.

Konfigurasi pada Android:

Daftarkan kunci API Android pada android/app/src/main/AndroidManifest.xml, di dalam tag <application> dan sebelum tag <activity>:

```
<manifest ...>
  <application ...>
    <meta-data
       android:name="com.google.android.geo.API_KEY"
       android:value="KUNCI_API_ANDROID_ANDA"/>
    ...
  </application>
</manifest>
```

Konfigurasi pada iOS:

Daftarkan kunci API iOS pada ios/Runner/*AppDelegate*.swift, di dalam metode application(_:didFinishLaunchingWithOptions:):

```
import Flutter
import UIKit
import GoogleMaps

@main
@objc class AppDelegate: FlutterAppDelegate, FlutterImplicitEngineDelegate {
  override func application(
    _ application: UIApplication,
    didFinishLaunchingWithOptions launchOptions: [UIApplication.LaunchOptionsKey:
Any]?
  ) -> Bool {
    GMSServices.provideAPIKey("KUNCI_API_IOS_ANDA")
    return super.application(application, didFinishLaunchingWithOptions: launchOptions)
  }

  func didInitializeImplicitFlutterEngine(_ engineBridge: FlutterImplicitEngineBridge) {
    GeneratedPluginRegistrant.register(with: engineBridge.pluginRegistry)
  }
}
```

Setelah *AppDelegate*.swift diperbarui, jalankan pod install pada folder ios/ (cd ios lalu pod install) agar CocoaPods menyinkronkan pustaka Google Maps untuk iOS sebelum aplikasi dijalankan ulang. Buat model data pendaftar terlebih dahulu, mengikuti pola *ChangeNotifier* yang sudah dipakai pada praktikum sebelumnya:

```
import 'package:flutter/foundation.dart';

class Pendaftar {
  final String nama;
  final String email;
  final String sesi;
  Pendaftar({required this.nama, required this.email, required this.sesi});
}
```

```
class PendaftarModel extends ChangeNotifier {
  final List<Pendaftar> _daftar = [];
  List<Pendaftar> get daftar => List.unmodifiable(_daftar);

  void tambah(Pendaftar pendaftar) {
    _daftar.add(pendaftar);
    notifyListeners();
  }

  int jumlahSesi(String sesi) => _daftar.where((p) => p.sesi == sesi).length;
}
```

Sediakan *PendaftarModel* melalui *ChangeNotifierProvider* (atau *MultiProvider* apabila digabung dengan model lain dari praktikum sebelumnya) di lib/main.dart, membungkus *MaterialApp*, agar seluruh halaman dapat mengaksesnya. Sesuaikan baris import sesuai lokasi berkas pada proyek Saudara:

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'model/pendaftar_model.dart';
import 'pages/formulir_page.dart';

void main() {
  runApp(
    ChangeNotifierProvider(
      create: (_) => PendaftarModel(),
      child: MaterialApp(
        title: 'Pendaftaran Kegiatan Kampus',
        home: const FormulirPage(),
      ),
    ),
  );
}
```

Dengan susunan ini, data pendaftar tidak lagi tersimpan sebagai state lokal yang hilang saat berpindah halaman, melainkan tersimpan pada *PendaftarModel* dan dapat dibaca ulang oleh halaman formulir maupun halaman statistik.

b. Membuat Form dengan *TextField*

Berkas: lib/pages/formulir_page.dart

Buat *FormulirPage* sebagai *StatefulWidget* dengan dua *TextEditingController* untuk menampung nama dan email peserta.

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
```

```
import '../model/pendaftar_model.dart';

class FormulirPage extends StatefulWidget {
  const FormulirPage({super.key});
  @override
  State<FormulirPage> createState() => _FormulirPageState();
}

class _FormulirPageState extends State<FormulirPage> {
  final namaController = TextEditingController();
  final emailController = TextEditingController();
  bool setujuSyarat = false;
  String? sesiTerpilih;
  bool sedangMengirim = false;

  bool get formLengkap =>
      namaController.text.isNotEmpty &&
      emailController.text.isNotEmpty &&
      setujuSyarat &&
      sesiTerpilih != null;

  @override
  void dispose() {
    namaController.dispose();
    emailController.dispose();
    super.dispose();
  }

  Future<void> kirimFormulir() async {
    setState(() => sedangMengirim = true);
    await Future.delayed(const Duration(seconds: 2));
    if (!mounted) return;
    context.read<PendaftarModel>().tambah(
      Pendaftar(
        nama: namaController.text,
        email: emailController.text,
        sesi: sesiTerpilih!,
      ),
```

```
  );
  setState(() => sedangMengirim = false);
  ScaffoldMessenger.of(context).showSnackBar(
   const SnackBar(content: Text('Pendaftaran berhasil dikirim')),
  );
 }

 @override
 Widget build(BuildContext context) {
  return Scaffold(
   appBar: AppBar(title: const Text('Formulir Pendaftaran')),
   floatingActionButton: FloatingActionButton(
    onPressed: () {
     setState(() {
      namaController.clear();
      emailController.clear();
      setujuSyarat = false;
      sesiTerpilih = null;
     });
    },
    tooltip: 'Bersihkan formulir',
    child: const Icon(Icons.refresh),
   ),
   body: SingleChildScrollView(
    padding: const EdgeInsets.all(16),
    child: Column(
     children: [
      GestureDetector(
       onTap: () => Navigator.pushNamed(context, '/detail-kegiatan'),
       onLongPress: () {
        ScaffoldMessenger.of(context).showSnackBar(
         const SnackBar(
          content: Text('Kegiatan: Seminar Teknologi Mobile'),
         ),
        );
       },
       child: Card(
        child: ListTile(
```

```
        leading: const Icon(Icons.event),
        title: const Text('Seminar Teknologi Mobile'),
        subtitle: const Text(
          'Ketuk untuk detail, tahan untuk info singkat',
        ),
      ),
    ),
  ),
),
const SizedBox(height: 12),
TextField(
  controller: namaController,
  decoration: const InputDecoration(
    labelText: 'Nama Lengkap',
    border: OutlineInputBorder(),
  ),
),
const SizedBox(height: 12),
TextField(
  controller: emailController,
  keyboardType: TextInputType.emailAddress,
  decoration: const InputDecoration(
    labelText: 'Email',
    border: OutlineInputBorder(),
  ),
),
CheckboxListTile(
  value: setujuSyarat,
  title: const Text(
    'Saya menyetujui syarat dan ketentuan kegiatan',
  ),
  onChanged: (nilai) {
    setState(() {
      setujuSyarat = nilai ?? false;
    });
  },
),
Column(
  children: ['Pagi', 'Siang', 'Sore'].map((sesi) {
```

```
            return RadioListTile<String>(
              title: Text('Sesi $sesi'),
              value: sesi,
              groupValue: sesiTerpilih,
              onChanged: (nilai) {
                setState(() {
                  sesiTerpilih = nilai;
                });
              },
            );
          }).toList(),
        ),
        const SizedBox(height: 12),
        ElevatedButton(
          onPressed: (formLengkap && !sedangMengirim)
              ? kirimFormulir
              : null,
          child: sedangMengirim
              ? const SizedBox(
                  height: 16,
                  width: 16,
                  child: CircularProgressIndicator(strokeWidth: 2),
                )
              : const Text('Kirim Pendaftaran'),
        ),
      ],
    ),
  ),
 );
 }
}
```

Controller wajib dibuang melalui *dispose*() agar tidak membebani memori setelah halaman ditutup. Nilai *controller*.text dibaca saat tombol kirim ditekan, bukan pada setiap ketikan, kecuali *onChanged* memang dibutuhkan untuk validasi langsung.



Gambar 4.1 Tangkapan layar formulir dengan TextField nama dan email

c. Menambahkan *Checkbox*

Berkas: lib/pages/formulir_page.dart

Tambahkan state lokal setujuSyarat pada _FormulirPageState, lalu sisipkan *CheckboxListTile* di dalam *Column* setelah kedua *TextField*. Berikut versi lengkap *FormulirPage* setelah penambahan ini:

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../model/pendaftar_model.dart';

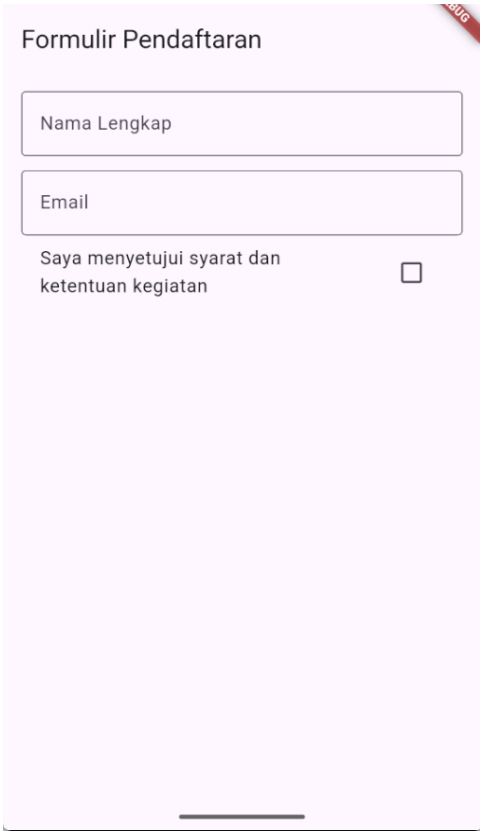
class FormulirPage extends StatefulWidget {
  const FormulirPage({super.key});
  @override
  State<FormulirPage> createState() => _FormulirPageState();
}

class _FormulirPageState extends State<FormulirPage> {
  final namaController = TextEditingController();
  final emailController = TextEditingController();
  bool setujuSyarat = false;

  @override
  void dispose() {
    namaController.dispose();
    emailController.dispose();
    super.dispose();
```

```
  }

  @override
  Widget build(BuildContext context) {
   return Scaffold(
     appBar: AppBar(title: const Text('Formulir Pendaftaran')),
     body: Padding(
       padding: const EdgeInsets.all(16),
       child: Column(
         children: [
           TextField(
             controller: namaController,
             decoration: const InputDecoration(
               labelText: 'Nama Lengkap',
               border: OutlineInputBorder(),
             ),
           ),
           const SizedBox(height: 12),
           TextField(
             controller: emailController,
             keyboardType: TextInputType.emailAddress,
             decoration: const InputDecoration(
               labelText: 'Email',
               border: OutlineInputBorder(),
             ),
           ),
           CheckboxListTile(
             value: setujuSyarat,
             title: const Text('Saya menyetujui syarat dan ketentuan kegiatan'),
             onChanged: (nilai) {
               setState(() {
                 setujuSyarat = nilai ?? false;
               });
             },
           ),
         ],
       ),
     ),
   );
  }
}
```



Gambar 4.2 Tangkapan layar Checkbox persetujuan pada formulir

d. Menambahkan *RadioButton*

Berkas: lib/pages/formulir_page.dart

Sediakan pilihan sesi kegiatan (Pagi, Siang, Sore) menggunakan satu variabel sesiTerpilih yang menjadi groupValue bagi ketiga *RadioListTile*. Berikut versi lengkap *FormulirPage* setelah penambahan ini:

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../model/pendaftar_model.dart';

class FormulirPage extends StatefulWidget {
  const FormulirPage({super.key});
  @override
  State<FormulirPage> createState() => _FormulirPageState();
}

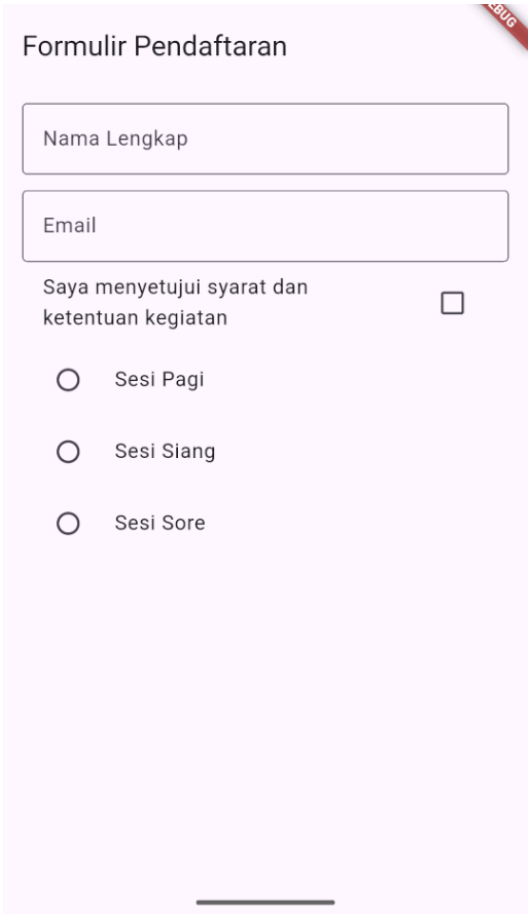
class _FormulirPageState extends State<FormulirPage> {
  final namaController = TextEditingController();
  final emailController = TextEditingController();
  bool setujuSyarat = false;
  String? sesiTerpilih;

  @override
  void dispose() {
    namaController.dispose();
    emailController.dispose();
```

```
  super.dispose();
 }

 @override
 Widget build(BuildContext context) {
  return Scaffold(
   appBar: AppBar(title: const Text('Formulir Pendaftaran')),
   body: Padding(
    padding: const EdgeInsets.all(16),
    child: Column(
     children: [
      TextField(
       controller: namaController,
       decoration: const InputDecoration(
        labelText: 'Nama Lengkap',
        border: OutlineInputBorder(),
       ),
      ),
      const SizedBox(height: 12),
      TextField(
       controller: emailController,
       keyboardType: TextInputType.emailAddress,
       decoration: const InputDecoration(
        labelText: 'Email',
        border: OutlineInputBorder(),
       ),
      ),
      CheckboxListTile(
       value: setujuSyarat,
       title: const Text('Saya menyetujui syarat dan ketentuan
kegiatan'),
       onChanged: (nilai) {
        setState(() {
         setujuSyarat = nilai ?? false;
        });
       },
      ),
      Column(
       children: ['Pagi', 'Siang', 'Sore'].map((sesi) {
        return RadioListTile<String>(
         title: Text('Sesi $sesi'),
         value: sesi,
         groupValue: sesiTerpilih,
         onChanged: (nilai) {
          setState(() {
           sesiTerpilih = nilai;
          });
         },
        );
       }).toList(),
      ),
     ],
```

```
        ),
      ),
    );
  }
}
```



Gambar 4.3 Tangkapan layar RadioButton pilihan sesi kunjungan

e. Menggunakan *ElevatedButton* untuk Mengirim Form

Berkas: lib/pages/formulir_page.dart

Tombol kirim dinonaktifkan (onPressed diberi null) apabila nama, email, atau persetujuan syarat belum lengkap. Saat proses pengiriman berjalan, tombol menampilkan indikator sedang mengirim melalui state lokal sedangMengirim, sedangkan data pendaftar yang berhasil dikirim disimpan ke *PendaftarModel* agar dapat dibaca oleh halaman lain. Berikut versi lengkap *FormulirPage* setelah penambahan ini:

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../model/pendaftar_model.dart';

class FormulirPage extends StatefulWidget {
  const FormulirPage({super.key});
  @override
  State<FormulirPage> createState() => _FormulirPageState();
```

```
}

class _FormulirPageState extends State<FormulirPage> {
  final namaController = TextEditingController();
  final emailController = TextEditingController();
  bool setujuSyarat = false;
  String? sesiTerpilih;
  bool sedangMengirim = false;

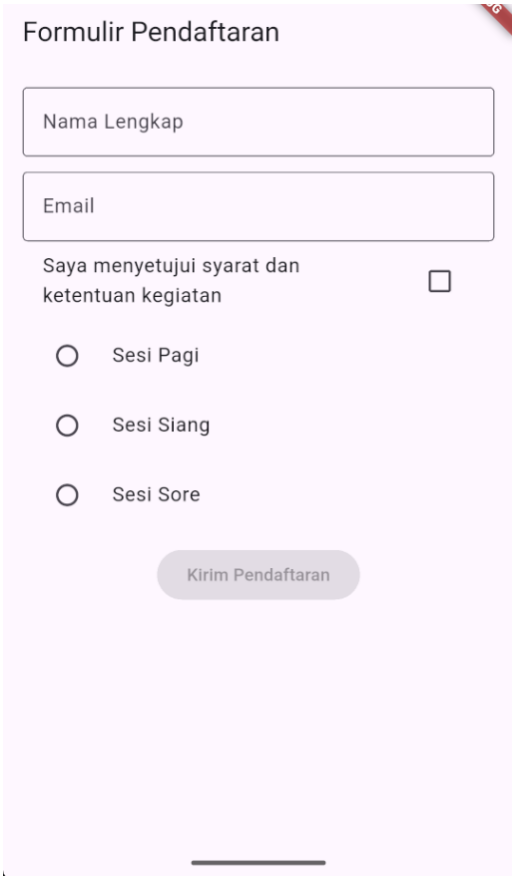
  bool get formLengkap =>
      namaController.text.isNotEmpty &&
      emailController.text.isNotEmpty &&
      setujuSyarat &&
      sesiTerpilih != null;

  @override
  void dispose() {
    namaController.dispose();
    emailController.dispose();
    super.dispose();
  }

  Future<void> kirimFormulir() async {
    setState(() => sedangMengirim = true);
    await Future.delayed(const Duration(seconds: 2));
    if (!mounted) return;
    context.read<PendaftarModel>().tambah(
      Pendaftar(nama: namaController.text, email: emailController.text, sesi:
sesiTerpilih!),
    );
    setState(() => sedangMengirim = false);
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Pendaftaran berhasil dikirim')),
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Formulir Pendaftaran')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [
            TextField(
              controller: namaController,
              decoration: const InputDecoration(
                labelText: 'Nama Lengkap',
                border: OutlineInputBorder(),
              ),
            ),
            const SizedBox(height: 12),
```

```
          TextField(
            controller: emailController,
            keyboardType: TextInputType.emailAddress,
            decoration: const InputDecoration(
              labelText: 'Email',
              border: OutlineInputBorder(),
            ),
          ),
          CheckboxListTile(
            value: setujuSyarat,
            title: const Text('Saya menyetujui syarat dan ketentuan kegiatan'),
            onChanged: (nilai) {
              setState(() {
                setujuSyarat = nilai ?? false;
              });
            },
          ),
          Column(
            children: ['Pagi', 'Siang', 'Sore'].map((sesi) {
              return RadioListTile<String>(
                title: Text('Sesi $sesi'),
                value: sesi,
                groupValue: sesiTerpilih,
                onChanged: (nilai) {
                  setState(() {
                    sesiTerpilih = nilai;
                  });
                },
              );
            }).toList(),
          ),
          const SizedBox(height: 12),
          ElevatedButton(
            onPressed: (formLengkap && !sedangMengirim) ? kirimFormulir :
null,
            child: sedangMengirim
                ? const SizedBox(
                    height: 16, width: 16,
                    child: CircularProgressIndicator(strokeWidth: 2),
                  )
                : const Text('Kirim Pendaftaran'),
          ),
        ],
      ),
    ),
  );
 }
}
```



Gambar 4.4 Tangkapan layar tombol Kirim Pengajuan beserta indikator memuat

f. Menambahkan *FloatingActionButton*

Berkas: lib/pages/formulir_page.dart

Tambahkan properti floatingActionButton pada *Scaffold* halaman formulir untuk mengosongkan seluruh isian dengan sekali tekan. Berikut versi lengkap *FormulirPage* setelah penambahan ini:

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../model/pendaftar_model.dart';

class FormulirPage extends StatefulWidget {
  const FormulirPage({super.key});
  @override
  State<FormulirPage> createState() => _FormulirPageState();
}

class _FormulirPageState extends State<FormulirPage> {
  final namaController = TextEditingController();
  final emailController = TextEditingController();
  bool setujuSyarat = false;
  String? sesiTerpilih;
  bool sedangMengirim = false;

  bool get formLengkap =>
      namaController.text.isNotEmpty &&
```

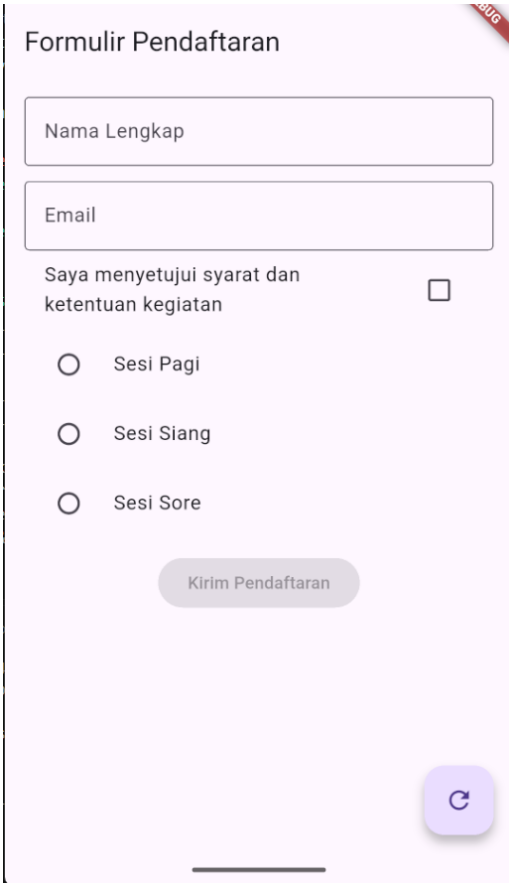
```
    emailController.text.isNotEmpty &&
    setujuSyarat &&
    sesiTerpilih != null;

 @override
 void dispose() {
  namaController.dispose();
  emailController.dispose();
  super.dispose();
 }

 Future<void> kirimFormulir() async {
  setState(() => sedangMengirim = true);
  await Future.delayed(const Duration(seconds: 2));
  if (!mounted) return;
  context.read<PendaftarModel>().tambah(
   Pendaftar(nama: namaController.text, email: emailController.text,
sesi: sesiTerpilih!),
  );
  setState(() => sedangMengirim = false);
  ScaffoldMessenger.of(context).showSnackBar(
   const SnackBar(content: Text('Pendaftaran berhasil dikirim')),
  );
 }

 @override
 Widget build(BuildContext context) {
  return Scaffold(
   appBar: AppBar(title: const Text('Formulir Pendaftaran')),
   floatingActionButton: FloatingActionButton(
    onPressed: () {
     setState(() {
      namaController.clear();
      emailController.clear();
      setujuSyarat = false;
      sesiTerpilih = null;
     });
    },
    tooltip: 'Bersihkan formulir',
    child: const Icon(Icons.refresh),
   ),
   body: Padding(
    padding: const EdgeInsets.all(16),
    child: Column(
     children: [
      TextField(
       controller: namaController,
       decoration: const InputDecoration(
        labelText: 'Nama Lengkap',
        border: OutlineInputBorder(),
       ),
      ),
```

```
          const SizedBox(height: 12),
          TextField(
            controller: emailController,
            keyboardType: TextInputType.emailAddress,
            decoration: const InputDecoration(
              labelText: 'Email',
              border: OutlineInputBorder(),
            ),
          ),
          CheckboxListTile(
            value: setujuSyarat,
            title: const Text('Saya menyetujui syarat dan ketentuan
kegiatan'),
            onChanged: (nilai) {
              setState(() {
                setujuSyarat = nilai ?? false;
              });
            },
          ),
          Column(
            children: ['Pagi', 'Siang', 'Sore'].map((sesi) {
              return RadioListTile<String>(
                title: Text('Sesi $sesi'),
                value: sesi,
                groupValue: sesiTerpilih,
                onChanged: (nilai) {
                  setState(() {
                    sesiTerpilih = nilai;
                  });
                },
              );
            }).toList(),
          ),
          const SizedBox(height: 12),
          ElevatedButton(
            onPressed: (formLengkap && !sedangMengirim) ?
kirimFormulir : null,
            child: sedangMengirim
                ? const SizedBox(
                    height: 16, width: 16,
                    child: CircularProgressIndicator(strokeWidth: 2),
                  )
                : const Text('Kirim Pendaftaran'),
          ),
        ],
      ),
    ),
  );
 }
}
```



Gambar 4.5 Tangkapan layar FloatingActionButton untuk mengatur ulang formulir

g. Menggunakan *GestureDetector* untuk Interaksi Kustom

Berkas: lib/pages/formulir_page.dart

Tampilkan kartu informasi kegiatan di bagian atas formulir. Sentuhan tunggal membuka halaman detail kegiatan, sedangkan sentuhan panjang menampilkan info singkat tanpa berpindah halaman. Berikut versi lengkap *FormulirPage* setelah penambahan ini (versi akhir halaman formulir):

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../model/pendaftar_model.dart';

class FormulirPage extends StatefulWidget {
  const FormulirPage({super.key});
  @override
  State<FormulirPage> createState() => _FormulirPageState();
}

class _FormulirPageState extends State<FormulirPage> {
  final namaController = TextEditingController();
  final emailController = TextEditingController();
  bool setujuSyarat = false;
  String? sesiTerpilih;
  bool sedangMengirim = false;
```

```
  bool get formLengkap =>
      namaController.text.isNotEmpty &&
      emailController.text.isNotEmpty &&
      setujuSyarat &&
      sesiTerpilih != null;

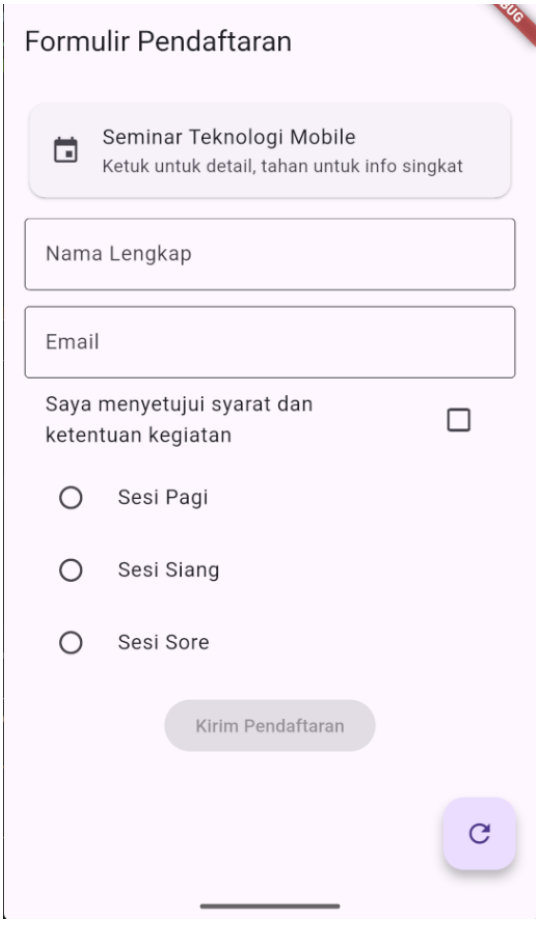
  @override
  void dispose() {
    namaController.dispose();
    emailController.dispose();
    super.dispose();
  }

  Future<void> kirimFormulir() async {
    setState(() => sedangMengirim = true);
    await Future.delayed(const Duration(seconds: 2));
    if (!mounted) return;
    context.read<PendaftarModel>().tambah(
      Pendaftar(nama: namaController.text, email:
emailController.text, sesi: sesiTerpilih!),
    );
    setState(() => sedangMengirim = false);
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Pendaftaran berhasil dikirim')),
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Formulir Pendaftaran')),
      floatingActionButton: FloatingActionButton(
        onPressed: () {
          setState(() {
            namaController.clear();
            emailController.clear();
            setujuSyarat = false;
            sesiTerpilih = null;
          });
        },
        tooltip: 'Bersihkan formulir',
        child: const Icon(Icons.refresh),
      ),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [
            GestureDetector(
              onTap: () => Navigator.pushNamed(context, '/detail-
kegiatan'),
              onLongPress: () {
                ScaffoldMessenger.of(context).showSnackBar(
```

```
              const SnackBar(content: Text('Kegiatan: Seminar
Teknologi Mobile')),
            );
           },
           child: Card(
            child: ListTile(
             leading: const Icon(Icons.event),
             title: const Text('Seminar Teknologi Mobile'),
             subtitle: const Text('Ketuk untuk detail, tahan untuk info
singkat'),
            ),
           ),
          ),
          const SizedBox(height: 12),
          TextField(
           controller: namaController,
           decoration: const InputDecoration(
            labelText: 'Nama Lengkap',
            border: OutlineInputBorder(),
           ),
          ),
          const SizedBox(height: 12),
          TextField(
           controller: emailController,
           keyboardType: TextInputType.emailAddress,
           decoration: const InputDecoration(
            labelText: 'Email',
            border: OutlineInputBorder(),
           ),
          ),
          CheckboxListTile(
           value: setujuSyarat,
           title: const Text('Saya menyetujui syarat dan ketentuan
kegiatan'),
           onChanged: (nilai) {
            setState(() {
             setujuSyarat = nilai ?? false;
            });
           },
          ),
          Column(
           children: ['Pagi', 'Siang', 'Sore'].map((sesi) {
            return RadioListTile<String>(
             title: Text('Sesi $sesi'),
             value: sesi,
             groupValue: sesiTerpilih,
             onChanged: (nilai) {
              setState(() {
               sesiTerpilih = nilai;
              });
             },
            );
```

```
            }).toList(),
          ),
          const SizedBox(height: 12),
          ElevatedButton(
            onPressed: (formLengkap && !sedangMengirim) ?
kirimFormulir : null,
            child: sedangMengirim
                ? const SizedBox(
                    height: 16, width: 16,
                    child: CircularProgressIndicator(strokeWidth: 2),
                  )
                : const Text('Kirim Pendaftaran'),
          ),
        ],
      ),
    ),
  );
 }
}
```



Gambar 4.6 Tangkapan layar interaksi GestureDetector pada kartu informasi kegiatan

h. Mengintegrasikan Pustaka *Chat* Sederhana

Berkas: lib/pages/chat_page.dart

Buat *ChatPage* yang menampilkan percakapan antara peserta dan panitia. Daftar pesan disimpan sebagai *state* lokal dan tampil terbalik agar pesan terbaru selalu berada di bagian bawah.

```
import 'package:flutter/material.dart';

class ChatPage extends StatefulWidget {
  const ChatPage({super.key});
  @override
  State<ChatPage> createState() => _ChatPageState();
}

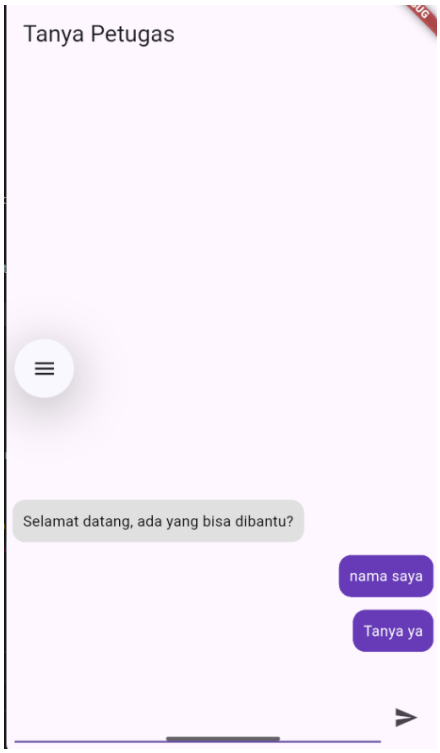
class _ChatPageState extends State<ChatPage> {
  final pesanController = TextEditingController();
  final List<Map<String, dynamic>> pesan = [
    {'teks': 'Selamat datang, ada yang bisa dibantu?', 'dariSaya': false},
  ];

  void kirimPesan() {
    if (pesanController.text.trim().isEmpty) return;
    setState(() {
      pesan.add({'teks': pesanController.text.trim(), 'dariSaya': true});
      pesanController.clear();
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Tanya Petugas')),
      body: Column(
        children: [
          Expanded(
            child: ListView.builder(
              reverse: true,
              itemCount: pesan.length,
              itemBuilder: (context, index) {
                final data = pesan[pesan.length - 1 - index];
                return Align(
                  alignment: data['dariSaya']
                      ? Alignment.centerRight
                      : Alignment.centerLeft,
                  child: Container(
                    margin: const EdgeInsets.all(6),
                    padding: const EdgeInsets.all(10),
                    decoration: BoxDecoration(
                      color: data['dariSaya'] ? Colors.deepPurple : Colors.grey[300],
                      borderRadius: BorderRadius.circular(12),
                    ),
                    child: Text(
```

```
                    data['teks'],
                    style: TextStyle(color: data['dariSaya'] ? Colors.white :
Colors.black),
                  ),
                ),
              );
            },
          ),
        ),
        Padding(
          padding: const EdgeInsets.all(8),
          child: Row(
            children: [
              Expanded(child: TextField(controller: pesanController)),
              IconButton(
                icon: const Icon(Icons.send),
                onPressed: kirimPesan,
              ),
            ],
          ),
        ),
      ],
    ),
  );
 }
}
```

Baris input diletakkan pada bagian bawah *Scaffold* melalui *Padding* dan *Row*, terdiri atas *TextField* beserta *IconButton* bergambar kirim yang memanggil kirimPesan().



Gambar 4.7 Tangkapan layar halaman chat sederhana

i. Mengintegrasikan Google Maps

Berkas: lib/pages/peta_page.dart

Buat *PetaPage* yang menampilkan lokasi kegiatan pada peta. *Widget GoogleMap* membutuhkan posisi kamera awal serta satu marker penanda gedung tempat kegiatan berlangsung.

```
import 'package:flutter/material.dart';
import 'package:google_maps_flutter/google_maps_flutter.dart';

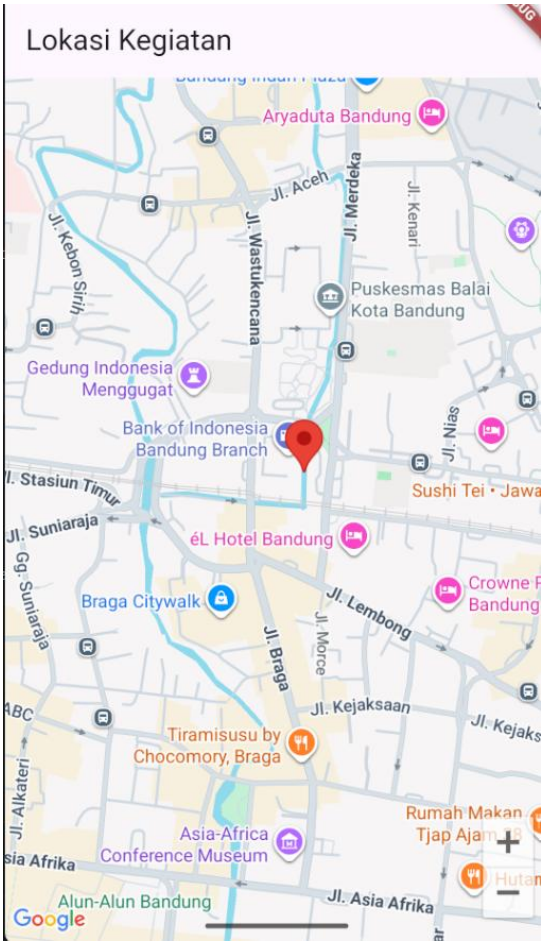
class PetaPage extends StatefulWidget {
  const PetaPage({super.key});
  @override
  State<PetaPage> createState() => _PetaPageState();
}

class _PetaPageState extends State<PetaPage> {
  static const LatLng lokasiKampus = LatLng(-6.914744, 107.609810);

  final Set<Marker> markers = {
    const Marker(
      markerId: MarkerId('gedung_kegiatan'),
      position: lokasiKampus,
      infoWindow: InfoWindow(title: 'Gedung Seminar Kampus'),
    ),
  };

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Lokasi Kegiatan')),
      body: GoogleMap(
        initialCameraPosition: const CameraPosition(
          target: lokasiKampus,
          zoom: 16,
        ),
        markers: markers,
      ),
    );
  }
}
```

Jalankan aplikasi pada *emulator* yang sudah memiliki Google Play Services, atau pada perangkat fisik, karena *widget GoogleMap* tidak akan menampilkan peta pada *emulator* tanpa Google Play Services.



Gambar 4.8 Tangkapan layar halaman peta dengan penanda lokasi

j. Mengintegrasikan Pustaka *Chart*

Berkas: lib/pages/statistik_page.dart

Halaman *StatistikPage* menampilkan jumlah pendaftar pada setiap sesi kunjungan (pagi, siang, sore) dalam bentuk *chart* batang menggunakan *fl_chart*, sehingga petugas dapat membaca sekilas sesi mana yang paling diminati tanpa menghitung satu per satu dari daftar pengajuan.

Tambahkan pustaka *fl_chart* terlebih dahulu:

```
flutter pub add fl_chart
```

Lengkapi *StatistikPage* sebagai *StatelessWidget* yang membaca *PendaftarModel* melalui *context.watch()*, menghitung jumlah pendaftar per sesi, lalu menampilkannya sebagai *BarChart*:

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'package:fl_chart/fl_chart.dart';
import '../model/pendaftar_model.dart';

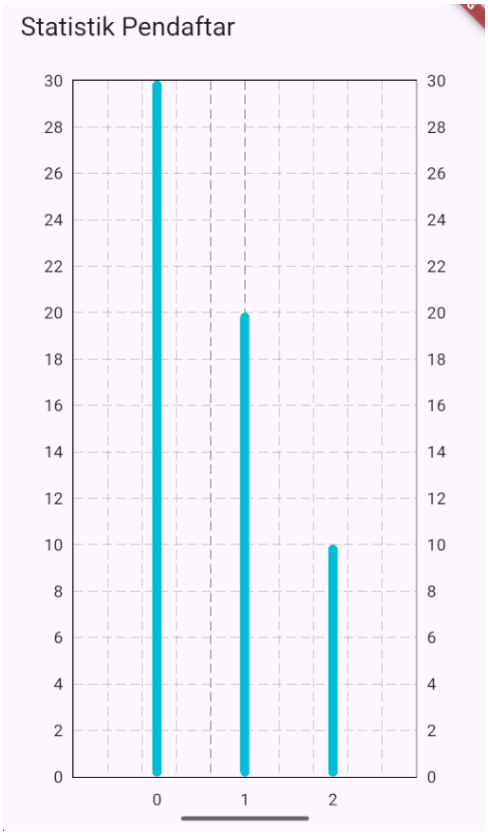
class StatistikPage extends StatelessWidget {
  const StatistikPage({super.key});

  @override
  Widget build(BuildContext context) {
    final model = context.watch<PendaftarModel>();
```

```
    final jumlahPagi = model.jumlahSesi('Pagi');
    final jumlahSiang = model.jumlahSesi('Siang');
    final jumlahSore = model.jumlahSesi('Sore');

    return Scaffold(
      appBar: AppBar(title: const Text('Statistik Pendaftar')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: BarChart(
          BarChartData(
            barGroups: [
              BarChartGroupData(x: 0, barRods:
[BarChartRodData(toY: jumlahPagi.toDouble())]),
              BarChartGroupData(x: 1, barRods:
[BarChartRodData(toY: jumlahSiang.toDouble())]),
              BarChartGroupData(x: 2, barRods:
[BarChartRodData(toY: jumlahSore.toDouble())]),
            ],
          ),
        ),
      ),
    );
  }
}
```

Karena *StatistikPage* memantau *PendaftarModel* melalui *context.watch()*, halaman ini akan tampil ulang secara otomatis setiap kali ada pendaftar baru yang dikirim dari *FormulirPage*, tanpa perlu penyegaran manual.



Gambar 4.9 Tangkapan layar halaman statistik pendaftar per sesi dalam bentuk chart batang

k. Menggabungkan Semua Fitur dengan Navigasi

Bagian ini mengikuti pola persis Modul II: named routes dipusatkan pada satu berkas bernama app_routes.dart, sedangkan navigasi antar halaman utama disatukan pada satu berkas kerangka navigasi bernama kerangka_navigasi.dart. *FormulirPage*, *ChatPage*, *PetaPage*, dan *StatistikPage* berperan seperti HalamanBeranda, HalamanJadwal, dan HalamanProfil pada Modul II, yaitu ditampilkan bergantian di dalam kerangka navigasi, bukan didaftarkan sebagai route tersendiri. Buat berkas **lib/navigation/app_routes.dart**, isinya kelas *AppRoutes* yang menyimpan nama route sebagai konstanta, persis seperti pola pada Modul II subbagian Navigasi dengan Named Routes:

```
import 'package:flutter/material.dart';
import '../pages/detail_kegiatan_page.dart';
import 'kerangka_navigasi.dart';

class AppRoutes {
  static const String beranda = '/';
  static const String detailKegiatan = '/detail-kegiatan';

  // Route tanpa argumen didaftarkan di sini.
  static Map<String, WidgetBuilder> daftarRoute() {
    return {
      beranda: (context) => const KerangkaNavigasi(),
      detailKegiatan: (context) => const DetailKegiatanPage(),
    };
  }

  // Disediakan untuk route yang memerlukan argumen di kemudian hari.
  static Route<dynamic>? bentukRoute(RouteSettings settings) {
    return null;
  }

  // Dipakai ketika nama route tidak terdaftar.
  static Route<dynamic> routeTidakDikenal(RouteSettings settings) {
    return MaterialPageRoute(
      builder: (context) => Scaffold(
        appBar: AppBar(title: const Text('Route Tidak Ditemukan')),
        body: Center(child: Text('Route ${settings.name} belum terdaftar.')),
      ),
    );
  }
}
```

Berkas: lib/pages/detail_kegiatan_page.dart

```
import 'package:flutter/material.dart';

class DetailKegiatanPage extends StatelessWidget {
  const DetailKegiatanPage({super.key});

  @override
```

```
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Detail Kegiatan')),
      body: const Padding(
        padding: EdgeInsets.all(16),
        child: Text('Seminar Teknologi Mobile dilaksanakan di Gedung
Seminar Kampus, pukul 09.00 hingga selesai.'),
      ),
    );
  }
}
```

Buat berkas **lib/navigation/kerangka_navigasi.dart**. Kelas *KerangkaNavigasi* menyimpan indeks tujuan aktif, menampilkan *NavigationBar* pada layar sempit, dan beralih ke *NavigationRail* pada layar lebar, mengikuti versi final kerangka_navigasi.dart pada Modul II:

```
import 'package:flutter/material.dart';
import '../pages/formulir_page.dart';
import '../pages/chat_page.dart';
import '../pages/peta_page.dart';
import '../pages/statistik_page.dart';

class KerangkaNavigasi extends StatefulWidget {
  const KerangkaNavigasi({super.key});
  @override
  State<KerangkaNavigasi> createState() => _KerangkaNavigasiState();
}

class _KerangkaNavigasiState extends State<KerangkaNavigasi> {
  int _indeksTerpilih = 0;

  final List<Widget> _halaman = const [
    FormulirPage(),
    ChatPage(),
    PetaPage(),
    StatistikPage(),
  ];
  final List<String> _judul = const ['Formulir', 'Chat', 'Peta', 'Statistik'];

  void _pilihTujuan(int indeks) {
    setState(() => _indeksTerpilih = indeks);
  }

  @override
  Widget build(BuildContext context) {
    final double lebar = MediaQuery.of(context).size.width;
    final bool layarLebar = lebar >= 600;

    return Scaffold(
      appBar: AppBar(title: Text(_judul[_indeksTerpilih])),
      body: layarLebar ? _tataLetakLebar() : _halaman[_indeksTerpilih],
```

```
      bottomNavigationBar: layarLebar ? null : _buatBilahBawah(),
    );
  }

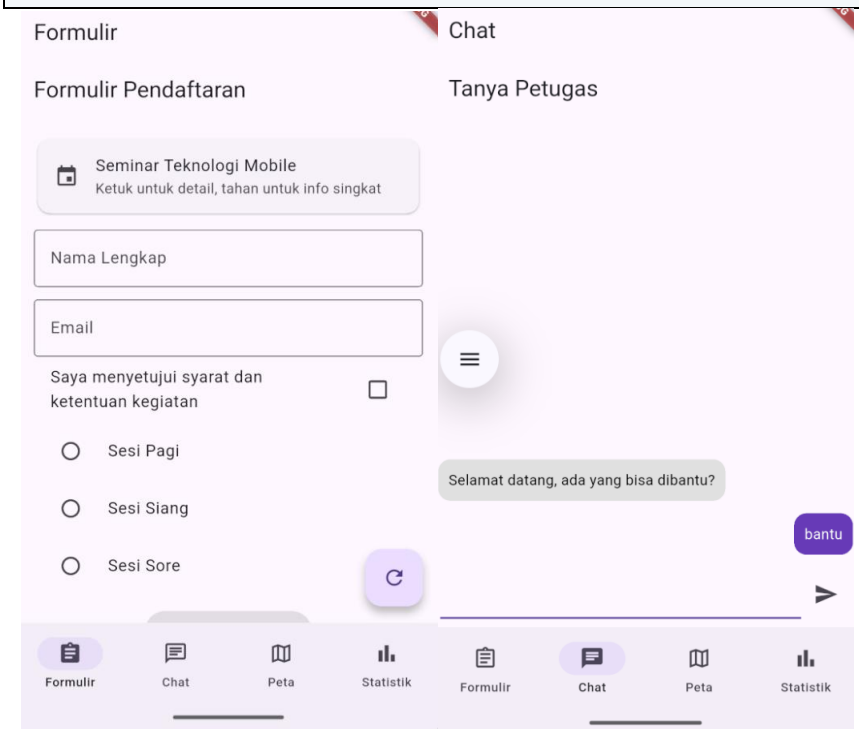
  // Bilah navigasi bawah, dipakai hanya pada layar sempit.
  Widget _buatBilahBawah() {
    return NavigationBar(
      selectedIndex: _indeksTerpilih,
      onDestinationSelected: _pilihTujuan,
      destinations: const [
        NavigationDestination(icon: Icon(Icons.assignment_outlined),
selectedIcon: Icon(Icons.assignment), label: 'Formulir'),
        NavigationDestination(icon: Icon(Icons.chat_outlined), selectedIcon:
Icon(Icons.chat), label: 'Chat'),
        NavigationDestination(icon: Icon(Icons.map_outlined), selectedIcon:
Icon(Icons.map), label: 'Peta'),
        NavigationDestination(icon: Icon(Icons.bar_chart_outlined),
selectedIcon: Icon(Icons.bar_chart), label: 'Statistik'),
      ],
    );
  }

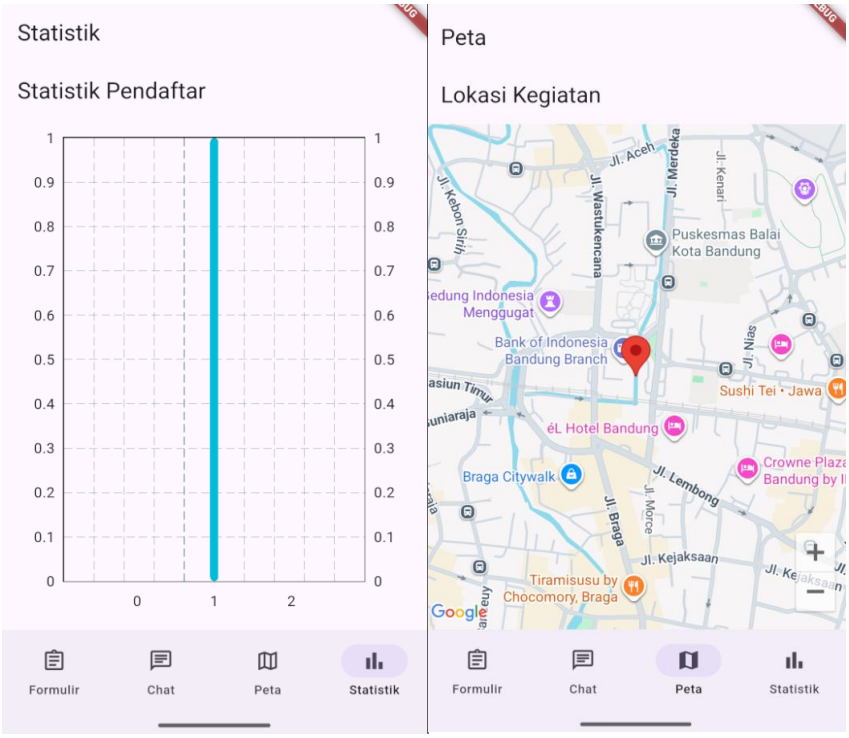
  // Tata letak layar lebar: rail di kiri, isi halaman di kanan.
  Widget _tataLetakLebar() {
    return Row(
      children: [
        NavigationRail(
          selectedIndex: _indeksTerpilih,
          onDestinationSelected: _pilihTujuan,
          extended: true,
          destinations: const [
            NavigationRailDestination(icon: Icon(Icons.assignment_outlined),
selectedIcon: Icon(Icons.assignment), label: Text('Formulir')),
            NavigationRailDestination(icon: Icon(Icons.chat_outlined),
selectedIcon: Icon(Icons.chat), label: Text('Chat')),
            NavigationRailDestination(icon: Icon(Icons.map_outlined),
selectedIcon: Icon(Icons.map), label: Text('Peta')),
            NavigationRailDestination(icon: Icon(Icons.bar_chart_outlined),
selectedIcon: Icon(Icons.bar_chart), label: Text('Statistik')),
          ],
        ),
        const VerticalDivider(thickness: 1, width: 1),
        Expanded(child: _halaman[_indeksTerpilih]),
      ],
    );
  }
}
```

Perbarui **lib/main.dart** agar memakai *AppRoutes*, sama seperti langkah terakhir pada Modul II:

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'model/pendaftar_model.dart';
import 'navigation/app_routes.dart';

void main() {
 runApp(
   ChangeNotifierProvider(
     create: (_) => PendaftarModel(),
     child: MaterialApp(
       title: 'Pendaftaran Kegiatan Kampus',
       initialRoute: AppRoutes.beranda,
       routes: AppRoutes.daftarRoute(),
       onGenerateRoute: AppRoutes.bentukRoute,
       onUnknownRoute: AppRoutes.routeTidakDikenal,
     ),
   ),
 );
}
```





Gambar 4.10 Tangkapan layar KerangkaNavigasi dengan NavigationBar dan NavigationRail adaptif

F. Materi Praktikum dengan Prompt

Bagian ini membangun Materi Praktikum dengan bantuan asisten AI sebagai tools, menggunakan proyek baru yang terpisah bernama form_interaksi_ai_nim agar hasil kerja manual pada bagian E dan hasil berbantuan AI dapat dibandingkan. Tujuannya bukan menggantikan proses belajar, melainkan melatih mahasiswa menulis kebutuhan dengan jelas, membaca hasil AI, menemukan kesalahan, dan menjelaskan kembali cara kerja kode. Setiap kode dari AI harus dibaca sebelum disalin, dijalankan, dibandingkan dengan ketentuan pada materi sebelumnya, dan diuji. Kode yang belum dapat dijelaskan mahasiswa tidak boleh dianggap selesai.

Tabel 5.1 Unsur Prompt yang Digunakan pada Praktikum Berbantuan AI

Unsur Prompt	Penjelasan
Peran	Menentukan siapa AI dan tingkat pembacanya, misalnya asisten Flutter untuk mahasiswa pemula.
Konteks	Menjelaskan nama proyek, materi, berkas yang sudah ada, dan tahap yang sedang dikerjakan.
Tugas	Menyebutkan satu hasil yang jelas, misalnya menulis satu berkas tertentu.
Batasan	Menjaga hasil tetap sesuai materi, paket yang diizinkan, dan struktur folder.
Format Keluaran	Menentukan apakah AI harus memberi rencana, kode lengkap, bagian yang berubah, atau penjelasan.
Verifikasi	Meminta AI menjelaskan alasan kode dan memberi daftar pemeriksaan yang dapat diuji mahasiswa.

a. Menyiapkan Proyek Praktikum dengan Bantuan AI

Tahap ini menghasilkan proyek baru.

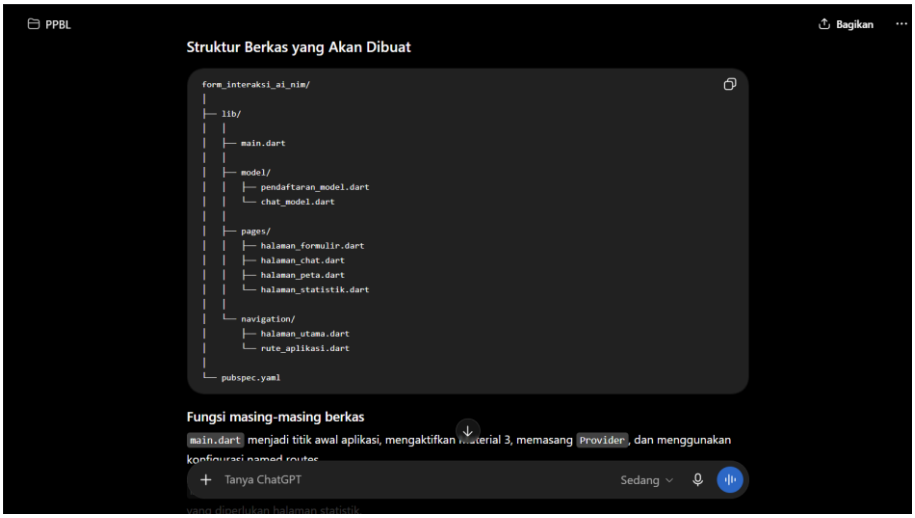
- Buka terminal Visual Studio Code, lalu jalankan perintah berikut. Ganti nim dengan nomor induk mahasiswa masing-masing.

```
flutter create form_interaksi_ai_nim
cd form_interaksi_ai_nim
flutter pub add provider
flutter pub add google_maps_flutter
flutter pub add fl_chart
code .
```

- Ketik prompt konteks awal berikut pada panel percakapan AI.

Kamu berperan sebagai asisten pemrograman Flutter untuk mahasiswa pemula.
Konteks: saya membuat proyek bernama form_interaksi_ai_nim di Visual Studio Code. Materi yang dipelajari adalah form dan input (*TextField*, *Checkbox*, *RadioButton*), tombol dan interaksi (*ElevatedButton*, *FloatingActionButton*, *GestureDetector*), pustaka chat, Google Maps, pustaka chart, serta navigasi dengan named routes.
Tujuan akhir: membantu saya membuat aplikasi Pendaftaran Kegiatan Kampus dengan halaman Formulir, Chat, Peta, dan Statistik yang dapat diakses melalui satu kerangka navigasi.
Batasan:
1. Gunakan Flutter Material 3 dan hanya package:flutter/material.dart, package:provider/provider.dart, package:*google_maps_flutter*/*google_maps_flutter*.dart, serta package:*fl_chart*/*fl_chart*.dart.
2. Pisahkan berkas model di lib/model, berkas halaman di lib/pages, dan berkas navigasi di lib/navigation.
3. Kerjakan bertahap. Jangan menulis semua kode sekaligus.
4. Setiap kode harus lengkap satu berkas, tidak boleh memakai tanda "..." untuk menghilangkan bagian kode.
5. Gunakan bahasa Indonesia dan nama kelas yang sama dengan nama berkas.
Format keluaran: tuliskan rencana kerja dan daftar berkas yang akan dibuat. Jangan menulis kode sebelum saya meminta tahap tertentu.

- Bandingkan rencana AI dengan struktur yang dipelajari pada materi di atas. Rencana sekurang-kurangnya memuat folder model, folder pages, folder navigation, dan berkas main.dart.
- Apabila AI langsung memberikan semua kode, jangan menyalinnya. Kirim ulang kalimat: "Ikuti format keluaran. Berikan rencana dan daftar berkas saja, tanpa kode."



Gambar 5.1 Rencana kerja proyek yang dihasilkan oleh AI

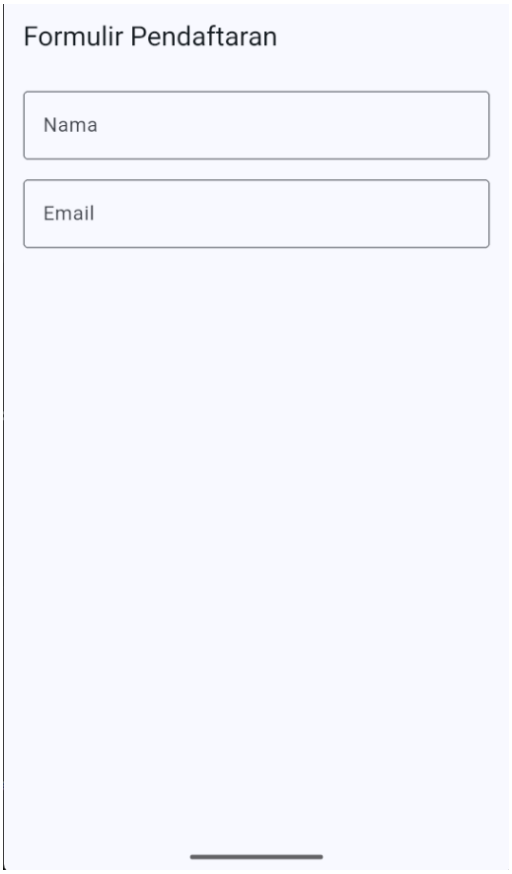
b. Membuat Form dan *TextField* dengan Bantuan AI

Tahap ini meminta AI membangun halaman formulir dengan dua *TextField*, yaitu nama dan email.

- Buat berkas kosong lib/pages/formulir_page.dart, lalu kirim prompt berikut dalam percakapan AI yang sama.

> Lanjutkan ke tahap membuat form dan textfield dengan bantuan ai.
> Tugas - lib/pages/formulir_page.dart:
> 1. Buat *StatefulWidget* bernama *FormulirPage*.
> 2. Sediakan *TextEditingController* terpisah untuk nama dan email.
> 3. Tampilkan dua *TextField* dengan *InputDecoration* berlabel, dan keyboardType email untuk kolom email.
> 4. Buang (*dispose*) kedua controller saat widget tidak lagi dipakai.
> Batasan: hanya gunakan package:flutter/material.dart. Tampilkan berkas secara lengkap dalam satu blok kode.
> Setelah kode, jelaskan mengapa controller wajib dibuang melalui *dispose*().

- Salin kode ke berkas yang sesuai, simpan, lalu lakukan hot restart.
- Bandingkan struktur kode dengan versi manual. Nama variabel boleh berbeda, tetapi kedua *TextField* dan *dispose*() harus tetap ada.
- Hasil yang harus diperiksa: kedua kolom dapat diisi, dan konsol tidak menunjukkan galat kebocoran memori saat halaman ditutup.



Gambar 5.2 Formulir dengan TextField nama dan email hasil AI

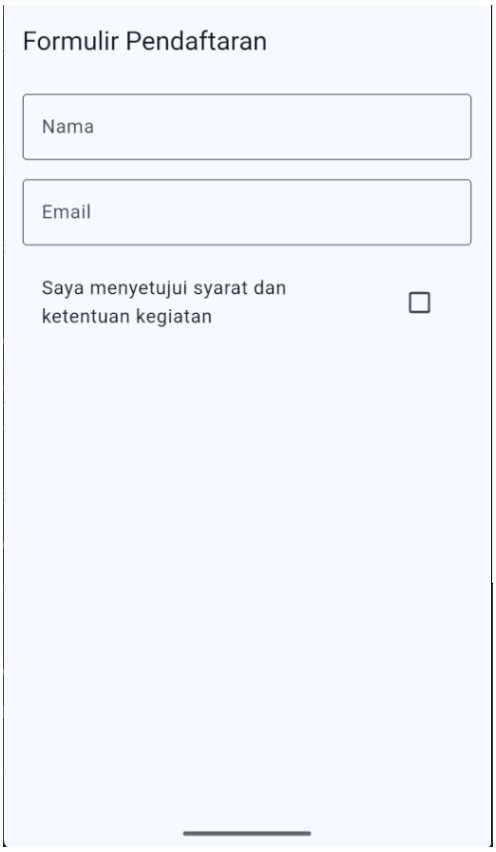
c. Menambahkan *Checkbox* dengan Bantuan AI

Tahap ini meminta AI menambahkan *Checkbox* persetujuan syarat dan ketentuan pada *FormulirPage* yang sudah ada.

- Buat berkas kosong lib/pages/formulir_page.dart, lalu kirim prompt berikut dalam percakapan AI yang sama.

Lanjutkan ke tahap menambahkan checkbox dengan bantuan ai. Tugas - lib/pages/formulir_page.dart: 1. Tambahkan variabel bool setujuSyarat mulai dari false pada _FormulirPageState. 2. Tambahkan *CheckboxListTile* dengan judul "Saya menyetujui syarat dan ketentuan kegiatan". 3. Perbarui setujuSyarat melalui *setState*() setiap kali checkbox ditekan. Batasan: jangan mengubah *TextField* yang sudah ada pada tahap sebelumnya. Setelah kode, jelaskan mengapa perubahan setujuSyarat harus ditulis di dalam *setState*().

- Salin kode ke berkas yang sesuai, simpan, lalu lakukan hot restart.
- Bandingkan dengan versi manual. variabel dan pesan boleh berbeda, tetapi logika *setState*() harus sama.
- Hasil yang harus diperiksa: checkbox berubah tercentang saat ditekan dan nilai setuju Syarat benar-benar berubah.



Gambar 5.3 Checkbox persetujuan pada formulir hasil AI

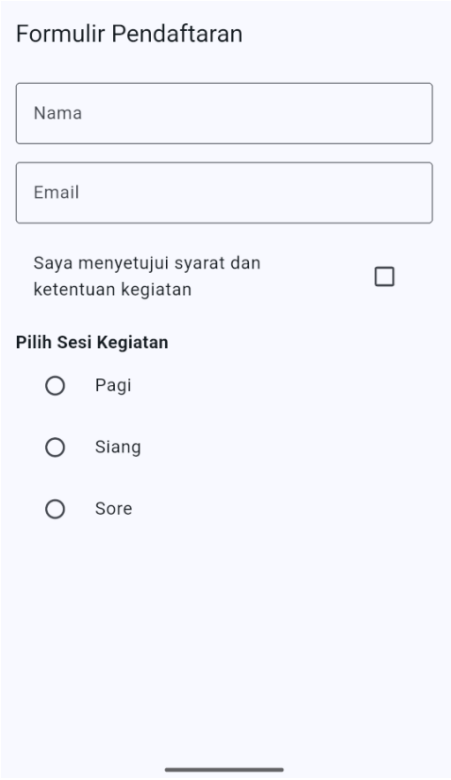
d. Menambahkan *RadioButton* dengan Bantuan AI

Tahap ini meminta AI menambahkan pilihan sesi kegiatan (Pagi, Siang, Sore) pada *FormulirPage*.

- Buat berkas kosong lib/pages/formulir_page.dart, lalu kirim prompt berikut dalam percakapan AI yang sama.

> Lanjutkan ke tahap menambahkan radiobutton dengan bantuan ai.
> Tugas - lib/pages/formulir_page.dart:
> 1. Tambahkan variabel String? sesiTerpilih mulai dari null.
> 2. Tampilkan tiga *RadioListTile*<String> untuk Pagi, Siang, dan Sore yang berbagi groupValue sesiTerpilih.
> 3. Perbarui sesiTerpilih melalui *setState*() setiap kali salah satu pilihan ditekan.
> Batasan: jangan mengubah *TextField* maupun *Checkbox* yang sudah ada.
> Setelah kode, jelaskan mengapa ketiga *RadioListTile* harus berbagi groupValue yang sama.

- Salin kode ke berkas yang sesuai, simpan, lalu lakukan hot restart.
- Bandingkan dengan versi manual.
- Hasil yang harus diperiksa: hanya satu pilihan yang dapat aktif pada satu waktu.



Gambar 5.4 RadioButton pilihan sesi kegiatan hasil AI

e. Menggunakan *ElevatedButton* dengan Bantuan AI

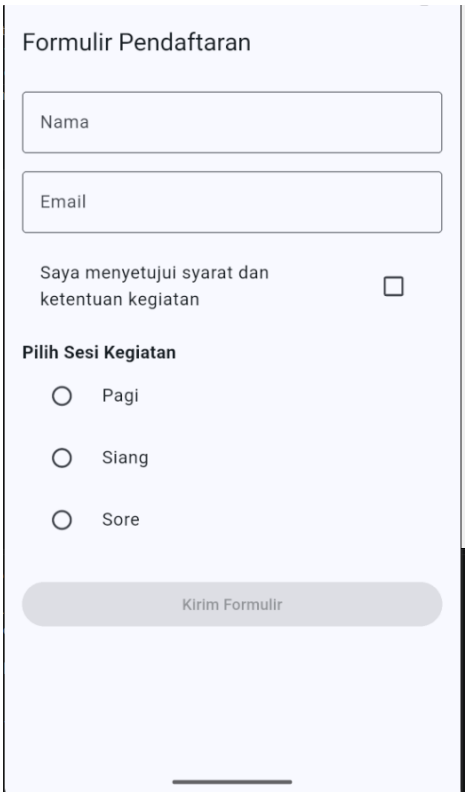
Tahap ini meminta AI membuatkan logika pengiriman formulir, termasuk validasi dan status sedang mengirim.

- Buat berkas kosong lib/pages/formulir_page.dart, lalu kirim prompt berikut dalam percakapan AI yang sama.

> Lanjutkan ke tahap menggunakan elevatedbutton dengan bantuan ai.
> Tugas - lib/pages/formulir_page.dart:
> 1. Tambahkan *ElevatedButton* berlabel “Kirim Formulir”.
> 2. Tombol nonaktif jika nama, email, atau setujuSyarat belum lengkap.

3. Saat ditekan, tampilkan status sedang mengirim selama dua detik menggunakan *setState*() dan *Future*.delayed, lalu tampilkan SnackBar "Formulir terkirim".
Batasan: belum perlu menyimpan data ke model apa pun pada tahap ini.
Setelah kode, jelaskan mengapa tombol harus tetap nonaktif selama status sedang mengirim aktif.

- Salin kode ke berkas yang sesuai, simpan, lalu lakukan hot restart.
- Bandingkan dengan versi manual.
- Hasil yang harus diperiksa: tombol tetap nonaktif walaupun ditekan berulang kali saat status sedang mengirim aktif.



Gambar 5.5 Tombol Kirim Formulir beserta indikator memuat hasil AI

f. Menambahkan *FloatingActionButton* dengan Bantuan AI

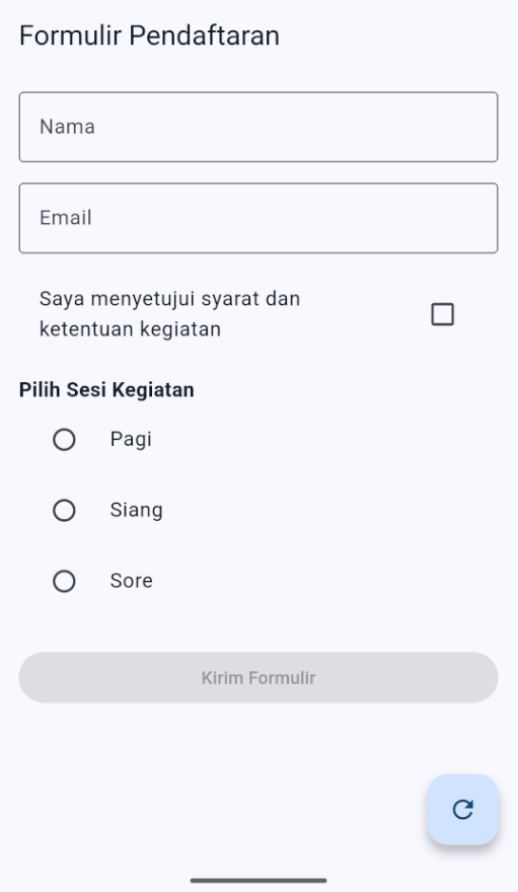
Tahap ini meminta AI menambahkan *FloatingActionButton* untuk mengosongkan seluruh isian formulir.

- Buat berkas kosong lib/pages/formulir_page.dart, lalu kirim prompt berikut dalam percakapan AI yang sama.

Lanjutkan ke tahap menambahkan floatingactionbutton dengan bantuan ai.
Tugas - lib/pages/formulir_page.dart:
1. Tambahkan *FloatingActionButton* berikon refresh pada Scaffold *FormulirPage*.
2. Saat ditekan, kosongkan namaController, emailController, setujuSyarat, dan sesiTerpilih ke kondisi awal melalui *setState*().
Batasan: pastikan controller.clear() dipanggil, bukan hanya mengganti variabel state.

Setelah kode, jelaskan mengapa controller.text tidak ikut kosong jika hanya variabel state yang direset.

- Salin kode ke berkas yang sesuai, simpan, lalu lakukan hot restart.
- Bandingkan dengan versi manual.
- Hasil yang harus diperiksa: seluruh isian benar-benar kosong setelah *FloatingActionButton* ditekan.



Gambar 5.6 FloatingActionButton pengatur ulang formulir hasil AI

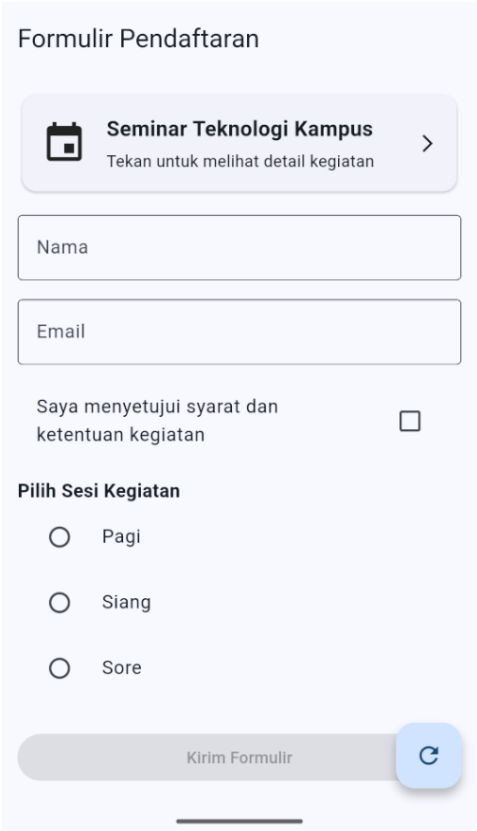
g. Menggunakan *GestureDetector* dengan Bantuan AI

Tahap ini meminta AI membuatkan kartu informasi kegiatan yang merespons ketukan dan tekan-tahan.

- Buat berkas kosong lib/pages/formulir_page.dart, lalu kirim prompt berikut dalam percakapan AI yang sama.

Lanjutkan ke tahap menggunakan gesturedetector dengan bantuan ai.
Tugas - lib/pages/formulir_page.dart:
1. Tambahkan Card informasi kegiatan di bagian atas Column, dibungkus *GestureDetector*.
2. onTap memanggil *Navigator*.pushNamed(*context*, '/detail-kegiatan').
3. onLongPress menampilkan SnackBar singkat berisi nama kegiatan.
Batasan: jelaskan juga perbedaan *GestureDetector* dengan *InkWell* untuk kasus ini.
Setelah kode, jelaskan mengapa *GestureDetector* tidak menampilkan efek riak seperti *InkWell*.

- Salin kode ke berkas yang sesuai, simpan, lalu lakukan hot restart.
- Bandingkan dengan versi manual.
- Hasil yang harus diperiksa: sentuhan tunggal membuka halaman detail dan sentuhan panjang menampilkan info singkat.



Gambar 5.7 Kartu informasi kegiatan dengan GestureDetector hasil AI

h. Mengintegrasikan Pustaka Chat dengan Bantuan AI

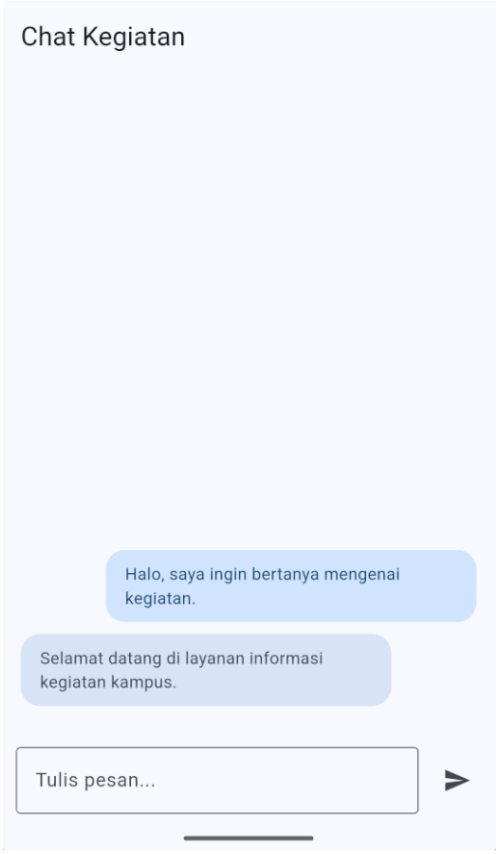
Tahap ini meminta AI membuatkan halaman chat sederhana sebagai sarana tanya jawab peserta dengan panitia kegiatan.

- Buat berkas kosong lib/pages/chat_page.dart, lalu kirim prompt berikut dalam percakapan AI yang sama.

> Lanjutkan ke tahap mengintegrasikan pustaka chat dengan bantuan ai.
> Tugas - lib/pages/chat_page.dart:
> 1. Buat *StatefulWidget* bernama *ChatPage* beserta wrapper *StatefulWidget*-nya.
> 2. Simpan daftar pesan sebagai List<Map<String,dynamic>>, tampilkan dengan ListView.builder(reverse: true).
> 3. Setiap pesan ditampilkan sebagai gelembung chat rata kanan atau kiri sesuai pengirim.
> 4. Tambahkan *TextField* dan IconButton kirim di bagian bawah yang memanggil *setState*() saat pesan ditambahkan.
> Batasan: tuliskan berkas secara lengkap termasuk class *ChatPage* extends *StatefulWidget*, jangan hanya class State-nya saja.
> Setelah kode, tanyakan kepada AI bagaimana caranya jika ingin mengganti

implementasi manual ini dengan pustaka pihak ketiga seperti *flutter_chat_ui*, lalu bandingkan jumlah kode yang diperlukan.

- Salin kode ke berkas yang sesuai, simpan, lalu lakukan hot restart.
- Bandingkan dengan versi manual.
- Hasil yang harus diperiksa: pesan baru yang dikirim langsung muncul di bagian bawah daftar dan kolom input kembali kosong.



Gambar 5.8 Halaman chat sederhana hasil AI

i. Mengintegrasikan Google Maps dengan Bantuan AI

Tahap ini meminta AI membuatkan halaman peta lokasi kegiatan menggunakan *google_maps_flutter*.

- Buat berkas kosong lib/pages/peta_page.dart, lalu kirim prompt berikut dalam percakapan AI yang sama.

Lanjutkan ke tahap mengintegrasikan google maps dengan bantuan ai.
Tugas - lib/pages/peta_page.dart:
1. Buat *StatefulWidget* bernama *PetaPage* beserta wrapper *StatefulWidget*-nya.
2. Tampilkan *GoogleMap* dengan initialCameraPosition menuju satu koordinat kampus.
3. Tambahkan satu *Marker* bertajuk “Lokasi Kegiatan” pada koordinat yang sama.
Batasan: jelaskan juga izin dan pengaturan tambahan apa saja yang diperlukan pada Android dan iOS agar peta dapat tampil.
Setelah kode, pastikan AI menyebutkan kebutuhan kunci API Google Maps serta Google Play Services pada emulator.

- Salin kode ke berkas yang sesuai, simpan, lalu lakukan hot restart.
- Bandingkan dengan versi manual.
- Hasil yang harus diperiksa: peta tampil dengan satu penanda pada lokasi yang benar, bukan layar abu-abu kosong.



Gambar 5.9 Halaman peta lokasi kegiatan hasil AI

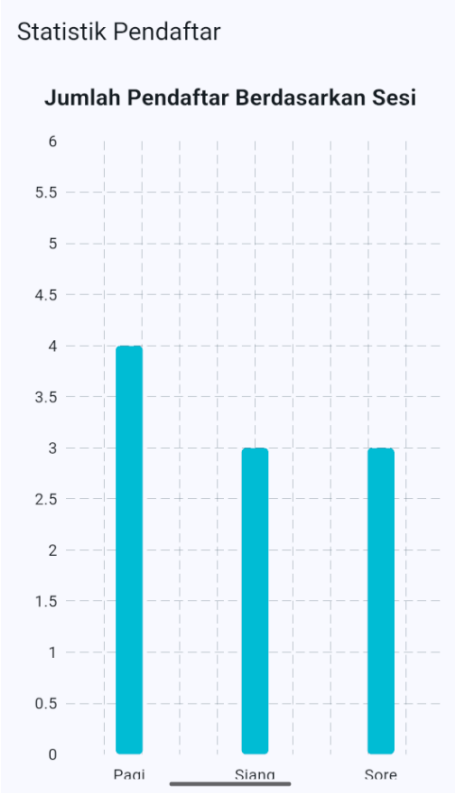
j. Mengintegrasikan Pustaka Chart dengan Bantuan AI

Tahap ini meminta AI membuat *StatistikPage* yang menampilkan jumlah pendaftar per sesi kegiatan dalam bentuk chart batang menggunakan *fl_chart*.

- Buat berkas kosong lib/pages/statistik_page.dart, lalu kirim prompt berikut dalam percakapan AI yang sama.

Lanjutkan ke tahap mengintegrasikan pustaka chart dengan bantuan ai. Tugas - lib/pages/statistik_page.dart: 1. Buat *StatelessWidget* bernama *StatistikPage*. 2. Hitung jumlah data untuk setiap sesi (Pagi, Siang, Sore) dari sumber data sederhana yang ditentukan sendiri oleh AI pada tahap ini. 3. Tampilkan hasilnya sebagai *BarChart* menggunakan *BarChartData* dan *BarChartGroupData*. Batasan: sertakan potongan kode pubspec.yaml yang perlu ditambahkan dan jelaskan setiap baris kode *BarChartData*. Setelah kode, pastikan AI menjelaskan bagaimana *BarChart* dapat diperbarui otomatis ketika data berubah.

- Salin kode ke berkas yang sesuai, simpan, lalu lakukan hot restart.
- Bandingkan dengan versi manual.
- Hasil yang harus diperiksa: chart batang tampil dengan tiga kelompok data sesuai jumlah sesi.



Gambar 5.10 Halaman statistik dalam bentuk chart batang hasil AI

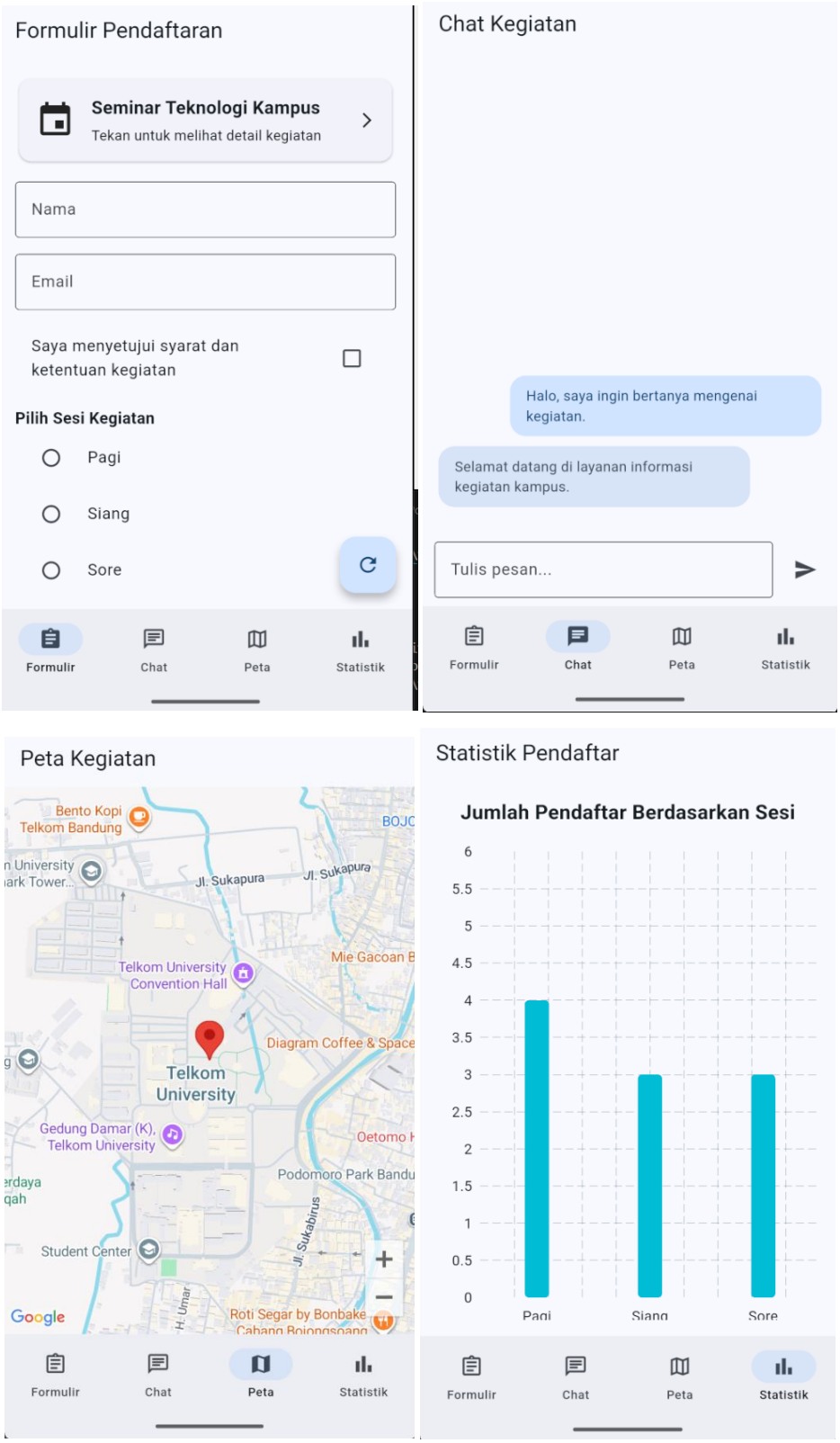
k. Menggabungkan Semua Fitur dengan Navigasi dengan Bantuan AI

Tahap ini meminta AI membuat kerangka navigasi terpusat mengikuti pola Modul II, yaitu memisahkan nama route pada app_routes.dart dan menempatkan bilah navigasi pada kerangka_navigasi.dart.

- Buat berkas kosong lib/navigation/app_routes.dart dan lib/navigation/kerangka_navigasi.dart, lalu kirim prompt berikut dalam percakapan AI yang sama.

```
Lanjutkan ke tahap menggabungkan semua fitur dengan navigasi dengan bantuan ai.
Tugas - lib/navigation/app_routes.dart dan lib/navigation/kerangka_navigasi.dart:
1. Buat lib/navigation/app_routes.dart berisi class AppRoutes dengan konstanta nama route
beranda dan detailKegiatan, metode daftarRoute(), bentukRoute(), dan routeTidakDikenal().
2. Buat lib/navigation/kerangka_navigasi.dart berisi StatefulWidget KerangkaNavigasi yang
menampilkan FormulirPage, ChatPage, PetaPage, dan StatistikPage bergantian melalui
NavigationBar pada layar sempit dan NavigationRail pada layar lebar (≥600).
3. Perbarui lib/main.dart agar memakai AppRoutes.
Batasan: buat berkas terpisah untuk setiap tujuan, jangan menggabungkan semuanya ke dalam
main.dart, dan sertakan lib/pages/detail_kegiatan_page.dart karena diimpor oleh
app_routes.dart.
Setelah kode, jelaskan alur main.dart membuka initialRoute, mencari nama route pada
AppRoutes.daftarRoute(), lalu menampilkan KerangkaNavigasi.
```

- Salin kode ke berkas yang sesuai, simpan, lalu lakukan hot restart.
- Bandingkan dengan versi manual.
- Hasil yang harus diperiksa: keempat halaman dapat dibuka bergantian melalui *NavigationBar*, dan berpindah ke *NavigationRail* ketika lebar layar diperbesar.



Gambar 5.11 KerangkaNavigasi dengan NavigationBar dan NavigationRail hasil AI

l. Menggabungkan, Menguji, dan Memperbaiki Galat dengan Bantuan AI

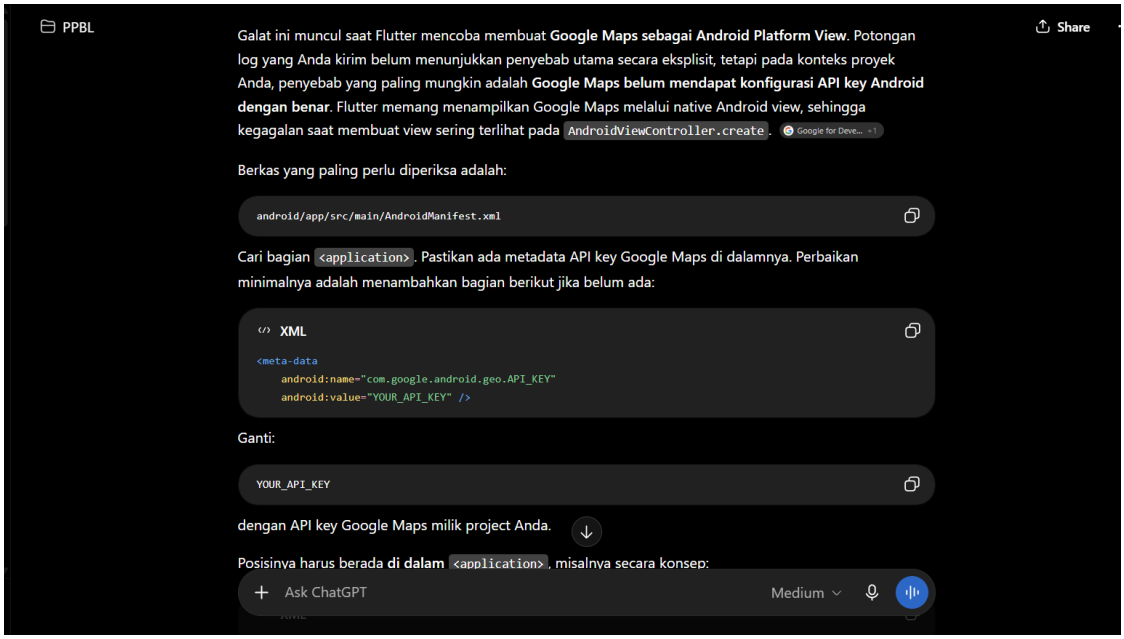
- Jalankan aplikasi form_interaksi_ai_nim secara utuh. Jika muncul galat, catat pesan galat lengkap, lalu kirim prompt berikut tanpa meminta AI menulis ulang seluruh berkas.

> Saya mendapatkan galat berikut saat menjalankan proyek Flutter form_interaksi_ai_nim:
> <tempelkan pesan galat lengkap di sini>
> Tugas kamu:
> 1. Jelaskan arti galat dengan bahasa sederhana.
> 2. Sebutkan nama berkas dan baris yang paling mungkin menjadi penyebab.
> 3. Tampilkan perbaikan minimal, hanya bagian kode yang berubah.
> 4. Jelaskan mengapa perbaikan tersebut menyelesaikan galat.
> Jangan menulis ulang seluruh berkas dan jangan mengubah fitur lain.

- Terapkan perbaikan secara manual, jalankan flutter analyze, lalu flutter run kembali.
- Hasil yang harus diperiksa: navigasi antara Formulir, Chat, Peta, dan Statistik berjalan, data pendaftar konsisten antara halaman formulir dan statistik, dan tidak ada galat saat aplikasi dijalankan.

Galat umum yang sering ditemui:

- MissingPluginException pada *GoogleMap*: biasanya muncul karena aplikasi belum dijalankan ulang penuh (bukan hot reload) setelah menambahkan pustaka native.
- Peta tampil kosong berwarna abu-abu: umumnya disebabkan kunci API belum diaktifkan atau belum ditempatkan pada berkas konfigurasi Android/iOS.
- RangeError pada *NavigationBar*/*NavigationRail*: jumlah destinations harus sama persis dengan jumlah halaman pada _halaman di kerangka_navigasi.dart.



Gambar 5.12 Tangkapan layar aplikasi form_interaksi_ai_nim gabungan setelah galat diperbaiki

m. Mengevaluasi Jawaban AI

Tahap penutup ini melatih mahasiswa menilai kode dari AI, bukan sekadar menyalinnya.

- Tanpa membuka kembali percakapan AI, jawab pertanyaan berikut menggunakan kata-kata sendiri: kapan sebaiknya memakai *setState*() dan kapan cukup memakai controller biasa; mengapa *dispose*() harus dipanggil untuk setiap *TextEditingController*; serta mengapa nama route sebaiknya disimpan sebagai konstanta pada *AppRoutes*, bukan ditulis berulang sebagai string.
- Pilih satu bagian kode dari AI yang belum sepenuhnya dipahami, lalu kirim prompt berikut tanpa meminta AI mengubah kodenya.

> Jelaskan potongan kode Flutter berikut baris demi baris untuk mahasiswa pemula. Jangan mengubah kodenya.
> <tempelkan potongan kode yang belum dipahami>
> Untuk setiap bagian, jelaskan:
> 1. Apa tugasnya.
> 2. Data apa yang masuk dan keluar.
> 3. Apa yang terjadi jika bagian itu dihapus.
> 4. Apakah bagian itu berhubungan dengan state lokal, *Provider*, atau navigasi.

G. Tugas Praktikum

Soal 1: Merancang dan Membangun Aplikasi Layanan Warga "Nusantara Cerdas Mobile"

Dinas Komunikasi dan Informatika berencana membangun aplikasi layanan warga bernama Nusantara Cerdas Mobile. Aplikasi ini dibutuhkan agar warga dapat mengajukan permohonan layanan, berkomunikasi dengan petugas, menemukan lokasi kantor layanan, serta memantau statistik pengajuan tanpa harus datang langsung ke balai kota. Kepala dinas menegaskan bahwa aplikasi harus dirancang dengan struktur navigasi yang jelas dan pengelolaan data yang konsisten sejak awal pengembangan, bukan disusun asal jalan.

Rancangan aplikasi mencakup enam bagian utama. Pertama, halaman rincian layanan memuat formulir pengajuan berisi kolom nama lengkap dan email (*TextField*), kotak persetujuan data pribadi (*Checkbox*), serta pilihan sesi kunjungan berupa pagi, siang, atau sore (*RadioButton*). Kedua, tombol Kirim Pengajuan (*ElevatedButton*) harus menampilkan indikator sedang mengirim sebelum data benar-benar tersimpan, dan tombol tidak aktif apabila formulir belum lengkap. Ketiga, tujuan Warga diberi *FloatingActionButton* untuk langsung membuka formulir pengajuan baru, serta setiap kartu riwayat pengajuan diberi *GestureDetector* agar dapat dibuka dengan sentuhan tunggal dan ditandai selesai ditindaklanjuti dengan sentuhan panjang.

Keempat, disediakan halaman Tanya Petugas berupa tampilan chat sederhana dan halaman Lokasi Kantor yang menampilkan peta menggunakan Google Maps dengan penanda pada setiap kantor layanan. Kelima, disediakan halaman Statistik Layanan yang menampilkan jumlah pengajuan pada setiap bidang layanan dalam bentuk chart batang. Keenam, seluruh halaman tersebut disatukan sejak awal ke dalam satu kerangka navigasi terpusat, mengikuti pola app_routes.dart dan kerangka_navigasi.dart, bukan ditambahkan secara terpisah-pisah setelah aplikasi jadi.

Ketentuan Pengerjaan

- Buat proyek Flutter baru bernama nusantara_cerdas_mobile_nim. Ganti nim dengan nomor induk mahasiswa masing-masing, lalu terapkan materi dari seluruh modul sebelumnya secara menyeluruh: gunakan *StatelessWidget* dan *StatefulWidget* dengan tata letak yang rapi,

susun navigasi antar halaman melalui app_routes.dart dan kerangka_navigasi.dart, kelola state lokal dengan *setState*() serta state bersama dengan *ChangeNotifier* dan *Provider*, kemudian lengkapi dengan form, tombol interaksi, chat, peta, dan chart.

- Rancang struktur folder lib/ sejak awal agar memuat folder navigation (berisi app_routes.dart dan kerangka_navigasi.dart), folder model (berisi seluruh model *ChangeNotifier*), dan folder pages (berisi seluruh halaman).
- Buat formulir pengajuan pada halaman rincian layanan menggunakan *TextField* dengan *TextEditingController* untuk nama dan email, dilengkapi validasi sederhana (email harus mengandung karakter "@").
- Tambahkan *Checkbox* persetujuan data pribadi; tombol Kirim Pengajuan hanya aktif ketika seluruh kolom terisi dan *Checkbox* tercentang.
- Tambahkan *RadioButton* atau *RadioListTile* untuk memilih sesi kunjungan, dengan nilai groupValue disimpan sebagai state lokal melalui *setState*().
- Rancang *PengajuanModel* sebagai class yang meng-extend *ChangeNotifier* untuk menyimpan seluruh data pengajuan, lalu sediakan melalui *ChangeNotifierProvider* di akar aplikasi. Pada *ElevatedButton* Kirim Pengajuan, terapkan status sedang mengirim (loading) selama proses simulasi pengiriman, lalu simpan hasil pengajuan ke *PengajuanModel* tersebut.
- Tambahkan *FloatingActionButton* pada tujuan Warga yang langsung membuka formulir pengajuan baru menggunakan named route, bukan *Navigator*.push langsung ke kelas halaman.
- Bungkus setiap kartu riwayat pengajuan dengan *GestureDetector*: onTap membuka rincian pengajuan, onLongPress menandai pengajuan sebagai selesai ditindaklanjuti melalui *PengajuanModel*.
- Buat halaman Tanya Petugas berisi daftar pesan (ListView.builder), gelembung pesan yang membedakan posisi warga dan petugas, kolom input, serta tombol kirim; boleh dibangun manual atau dengan bantuan pustaka chat pihak ketiga.
- Buat halaman Lokasi Kantor menggunakan *google_maps_flutter*, menampilkan minimal tiga penanda kantor layanan beserta *InfoWindow* nama kantor.
- Buat halaman Statistik Layanan yang membaca jumlah pengajuan pada setiap bidang layanan dari *PengajuanModel* melalui *context*.watch(), lalu menampilkannya sebagai *BarChart* menggunakan pustaka *fl_chart*.
- Daftarkan seluruh halaman (rincian layanan, Warga, Tanya Petugas, Lokasi Kantor, dan Statistik Layanan) sebagai tujuan pada kerangka_navigasi.dart sejak rancangan awal, dengan nama route yang didaftarkan sebagai konstanta pada app_routes.dart.
- Jalankan aplikasi pada emulator atau perangkat fisik, isi formulir pengajuan, kirim beberapa pesan pada halaman Tanya Petugas, buka halaman Lokasi Kantor, dan buka halaman Statistik Layanan, lalu ambil tangkapan layar setiap tahap.

Luaran yang Dikumpulkan

- Kode program pada repositori Git.
- Laporan dalam format PDF yang dikumpulkan melalui LMS.
- Tangkapan layar formulir pengajuan sebelum dan sesudah diisi, tombol Kirim Pengajuan saat menampilkan indikator memuat, kartu riwayat pengajuan yang ditandai selesai melalui sentuhan panjang, percakapan pada halaman Tanya Petugas, peta pada halaman Lokasi Kantor dengan penanda kantor layanan, serta chart batang pada halaman Statistik Layanan.

- Penjelasan singkat pada laporan mengenai rancangan struktur folder lib/navigation, lib/model, dan lib/pages yang dibuat, alasan pemilihan *TextField*, *Checkbox*, dan *RadioButton* pada formulir, serta alasan penggunaan *GestureDetector* dibandingkan *InkWell* pada kartu riwayat pengajuan.
- Berkas laporan diberi nama Modul4_NIM_NamaLengkap.pdf dan diunggah melalui LMS sesuai tenggat yang ditentukan.

Soal 2: Merancang dan Mengintegrasikan Sebagian Materi Modul Ini pada Rancangan Tugas Akhir Saudara

Rancang bagian yang dipilih pada tugas akhir Saudara seolah-olah dibangun dari awal: mulai dari struktur folder, model data, dan jalur navigasinya, bukan sekadar menempelkan widget baru pada halaman yang sudah ada.

Ketentuan Pengerjaan

- Tentukan nama aplikasi tugas akhir Saudara, lalu buat atau susun ulang proyek Flutter bernama tubes_namaaplikasi_nim sesuai rancangan tersebut.
- Pilih minimal dua materi berikut untuk dirancang pada aplikasi Saudara: (1) Form dan Input (*TextField*, *Checkbox*, dan/atau *RadioButton*), (2) Tombol dan Interaksi (*ElevatedButton*, *FloatingActionButton*, dan/atau *GestureDetector*), (3) Pustaka Chat, (4) Google Maps, (5) Pustaka Chart.
- Rancang struktur folder lib/ sejak awal agar memuat folder navigation (berisi app_routes.dart dan kerangka_navigasi.dart, atau berkas navigasi terpusat yang setara), folder model (berisi seluruh model *ChangeNotifier*), dan folder pages (berisi seluruh halaman).
- Untuk setiap materi yang dipilih, rancang model *ChangeNotifier* yang sesuai di folder lib/model, lalu sediakan melalui *ChangeNotifierProvider* atau *MultiProvider* di akar aplikasi; state tidak boleh hanya disimpan secara lokal melalui *setState*() saja.
- Rancang setiap halaman baru di folder lib/pages, lalu daftarkan sebagai route pada kerangka navigasi yang dibuat, dengan nama route yang didaftarkan sebagai konstanta pada berkas navigasi.
- Pastikan data yang dikelola melalui *Provider* benar-benar berubah pada tampilan (bukan nilai tetap) dan dapat dibaca dari minimal dua halaman berbeda apabila relevan dengan materi yang dipilih.
- Jalankan aplikasi dari awal hingga akhir dan pastikan tidak ada galat pada seluruh halaman yang mengakses *Provider* maupun navigasi yang baru dirancang.

Luaran yang Dikumpulkan

- Kode program pada repositori Git.
- Tangkapan layar setiap materi yang dipilih sebelum dan sesudah digunakan.
- Laporan Modul4_Soal2_NIM_NamaLengkap.pdf diunggah melalui LMS, menjelaskan materi yang dipilih, rancangan model *Provider* yang dibuat, serta struktur folder lib/navigation, lib/model, dan lib/pages pada proyek Saudara.

H. Format Pengumpulan Tugas

Laporan disusun secara berurutan dengan sistematika berikut.

1. **Halaman Sampul (Cover)**
   Memuat judul praktikum, nama mahasiswa, NIM, kelas, mata kuliah, nama dosen pengampu, program studi, serta tahun akademik. Gunakan logo institusi dan tata letak yang rapi.
2. **Tujuan Praktikum**
   Menuliskan tujuan yang ingin dicapai pada praktikum ini dengan kalimat sendiri, bukan menyalin ulang tujuan pembelajaran pada modul.
3. **Alat dan Bahan**
   Menyebutkan spesifikasi perangkat keras yang digunakan serta perangkat lunak beserta versinya, misalnya Flutter SDK, Dart SDK, Android Studio, Visual Studio Code, dan Git.
4. **Langkah-Langkah Praktikum**
   Menguraikan tahapan pengerjaan secara runtut mulai dari pembuatan proyek, pembuatan stateless widget, stateful widget, penyusunan layout dan styling, hingga penggabungan pada main.dart. Setiap langkah disertai potongan kode yang relevan dan penjelasan singkat.
5. **Hasil dan Dokumentasi Program**
   Menampilkan tangkapan layar hasil aplikasi yang berjalan, sekurang-kurangnya tampilan awal dan tampilan setelah terjadi perubahan state. Setiap gambar diberi nomor, keterangan gambar, dan penjelasan singkat.
6. **Analisis dan Pembahasan**
   Membahas alasan pemilihan stateless widget dan stateful widget pada masing-masing bagian aplikasi, kendala atau galat yang ditemukan beserta cara penyelesaiannya, serta keterkaitan aplikasi dengan pilar smart city yang diangkat.
7. **Kesimpulan**
   Menuliskan ringkasan capaian praktikum dan pemahaman yang diperoleh, disusun dalam beberapa kalimat yang menjawab tujuan praktikum.
8. **Lampiran berisi Link Repositori GitHub**
   Mencantumkan tautan repositori GitHub yang dapat diakses oleh dosen pengampu, memuat kode program lengkap.
9. **Presentasi Hasil Praktikum**
   Mahasiswa mempresentasikan hasil praktikum di kelas, meliputi demonstrasi aplikasi yang berjalan dan penjelasan kode program, serta menjawab pertanyaan dari dosen pengampu.